Advanced Web Development

5 - Custom Credentials, JWT, Sessions Handling, Robust Architecture for full stack integration

Jeannette Chin

Recall last weeks

- REACT Basics – component, JSX, rendering
- Event Handling and State Management in REACT - useState hook
- Next.js Basics
  - Routing and Navigation
  - Client and Server Components
  - Data Fetching – Fetch and Axios API
- useEffect hook
- RTK for global state management
- Web Authentication methods
- Google Provider with Auth.js
- Prisma ORM

Learning Objectives

- Understanding Web App Security, Risks and Defence Techniques
- Implement conventional password-based credentials
- HttpOnly Session Cookie
- Examine JWT
- Examine Sessions Handling with Best Practice
- Implement Credentials Provider by Auth.js
- Full-Stack integration
- Securing end points & implement security defence strategy

Recall Credentials System

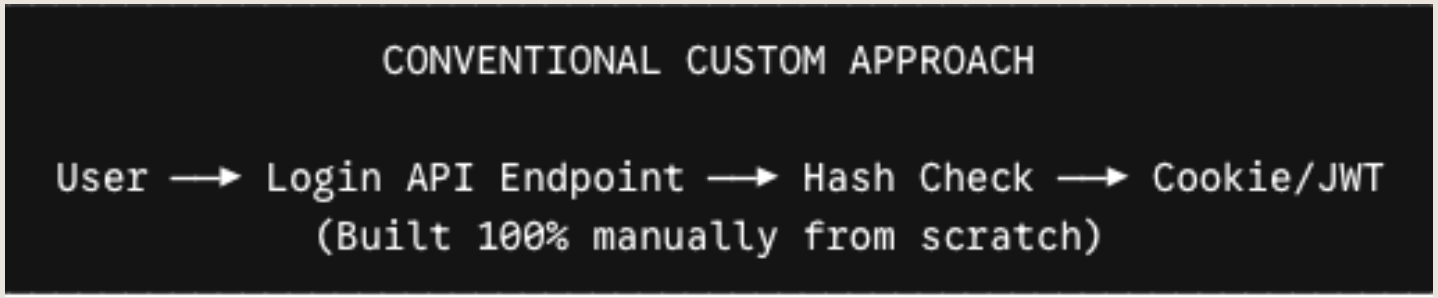
- A **Credentials System** is an authentication pattern where an application verifies a user's identity directly using credentials they supply (typically email/username + password, 2FA tokens, or API keys) against a custom user database or internal backend.
- This stands in contrast to **Delegated Authentication** (OAuth 2.0 / OIDC), where identity management is offloaded to an external provider like Google, GitHub, or Okta.

- We have explored Delegated Authentication using Google Provider last week. We will investigate conventional approaches this week!

Conventional Approach to implement Credentials System

Conventional Credentials System

- **Conventional Approach:** The engineering team builds every layer manually:
  - writing endpoint handlers
  - database lookups,
  - password hashing algorithms (e.g., Argon2 or bcrypt),
  - JWT signing/verification,
  - raw HTTP cookie headers (HttpOnly, SameSite, Secure).



Auth.js (Credentials Provider)

- `Auth.js` supports custom credentials system as well
- You provide a custom `authorize()` callback function to check credentials against your database/API, while Auth.js automatically manages session token creation, CSRF protection, cookie lifecycle, and frontend session hooks (well almost!).
- It uses JSON Web Tokens (JWT) for session management by default and does not automatically persist credential-authenticated users into database adapters.
- The `Auth.js` maintainers deliberately limit this to discourage raw password authentication over battle-tested OAuth or Passkeys due to the security overhead involved.

Advantages of Conventional Approach

- **Data Sovereignty:** You own 100% of the user identity lifecycle without dependency on external third-party uptime or pricing.
- **Legacy System Support:** Essential when integrating with existing enterprise databases, LDAP servers, or legacy user stores.

Disadvantages of Conventional Approach

- **High Maintenance Burden:** You are fully responsible for password resets, email verifications, account lockouts, and rate limiting.

- **Regulatory Responsibility:** Storing password hashes increases compliance scope (GDPR, SOC2, PCI-DSS) and risk of credential leaks

Conventional vs Auth.js

Dimension	Conventional Custom Auth	Auth.js (`CredentialsProvider`)
Control & Flexibility	**100% control** over schema, session storage, DB persistence, and custom token refresh logic.	**Constrained.** Use JWT sessions by default, user data isn't auto-synced to database adapters. Although it also supports DB sessions as well
Development Speed	**Slow.** Requires building validation, hashing, sessions, cookie parsing, and route guards.	**Fast.** Boilerplate for session handling, cookies, and CSRF protection is pre-built.
Multi-Provider Flexibility	**Harder.** Adding OAuth or SAML later requires writing brand-new authentication flows.	**Seamless.** Easily mix credentials with Google, GitHub, or Passkeys under one unified session hook (`useSession`).
Security Risk Profile	**Extreme.** High probability of subtle human errors in cookie settings, CSRF, or timing attacks.	**Medium-Low.** Provides safe defaults for cookies, CSRF, and state management.

Security Risks

When implementing custom credentials, we take on significant security vulnerabilities if proper controls are neglected:

Cryptographic & Hash Storage Failures

- Using outdated hashing algorithms (MD5, SHA1, plain SHA-256) or weak work factors.
- Storing un-salted hashes, enabling fast reverse lookups via pre-computed tables.

Security Risks

Brute Force & Credential Stuffing Attacks

- Automated bots testing millions of leaked username/password pairs against unprotected login endpoints.

Session & Cookie Misconfigurations

- Storing JWTs in `localStorage` or `sessionStorage`, leaving them exposed to **Cross-Site Scripting (XSS)**.
- Omitting the `HttpOnly` flag (allowing JavaScript token access) or `SameSite` attribute (exposing routes to **CSRF** attacks).

Security Risks

User Enumeration via Timing Attacks

- Returning an immediate error when an email doesn't exist, but taking 200ms to verify a password hash when it *does* exist. Attackers measure response delays to harvest valid email lists.

Lack of Identity Protections

- Failing to implement account lockout policies, multi-factor authentication (MFA), or breach detection.

Defensive Solutions & Best Practices

Modern Password Hashing

- Use memory-hard key derivation functions like **Argon2id** or **bcrypt** with appropriate cost factors (e.g., bcrypt salt rounds $\geq 10 - 12$). Avoid fast hash functions like SHA-256 for password storage.

Secure Token Storage & Cookie Hardening

- Store tokens exclusively in **`HTTP-only`, `Secure`, `SameSite=Lax (or Strict)`** cookies to prevent XSS access and CSRF exploitation.
- For `Auth.js`: Ensure a NEXTAUTH_SECRET is defined in production and never committed to source control.

Defensive Solutions & Best Practices

Rate Limiting & Bot Protection

- Apply strict rate limiting on authentication routes (e.g., max 5 attempts per minute per IP/account using Redis or sliding window algorithms).

- Add CAPTCHA or invisible bot protection (Cloudflare Turnstile) on sign-in and sign-up forms.

Defensive Solutions & Best Practices

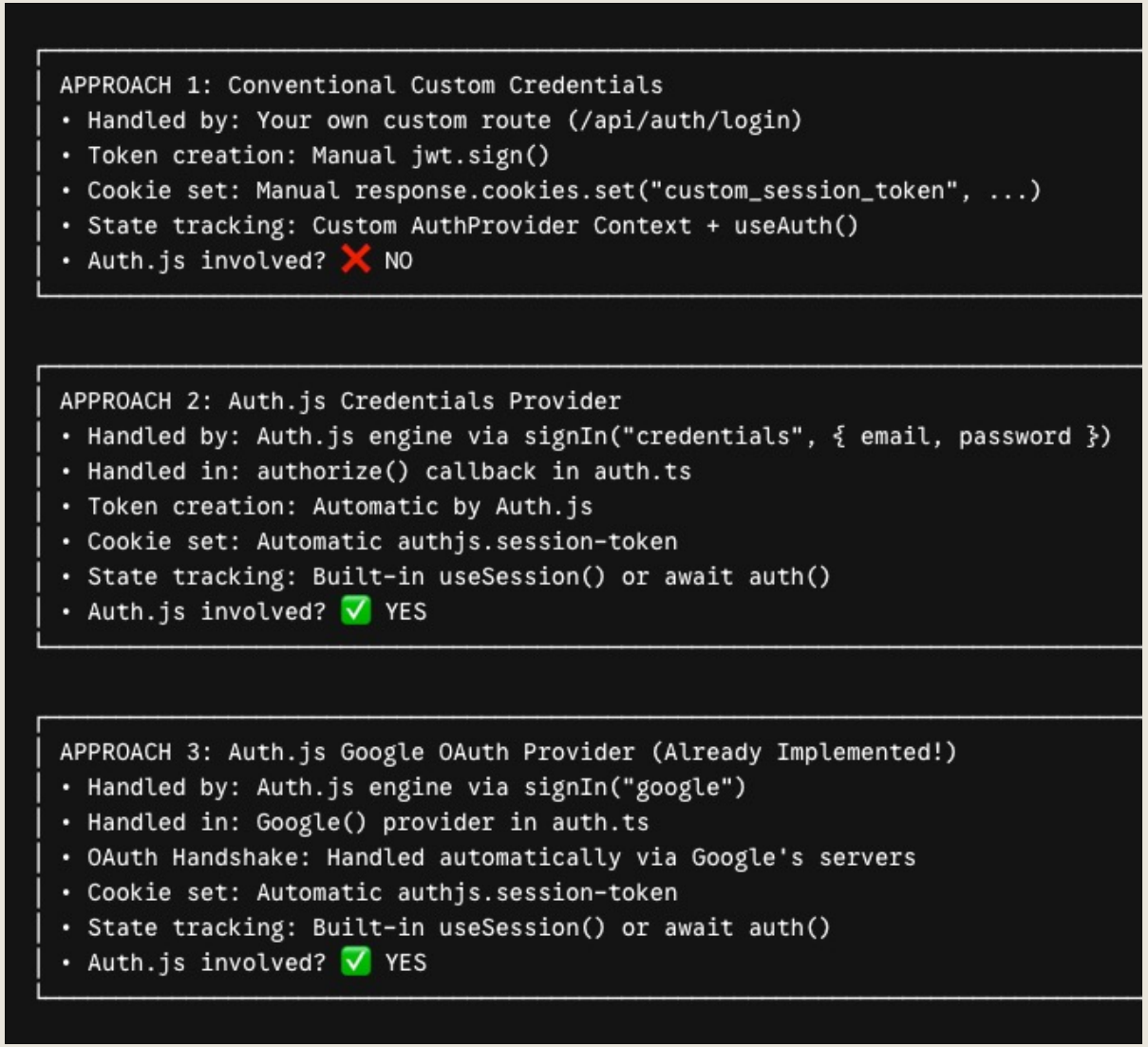
Constant-Time Authentication

- Prevent user enumeration by standardising response times.
- If an account isn't found, execute a dummy password hash verification to ensure execution time remains identical regardless of user existence.

Multi-Factor Authentication (MFA) & Upgrade Paths

- Layer credentials with Time-based One-Time Passwords (TOTP via Authenticator apps) or WebAuthn/Passkeys.

The Three Approaches



Custom Registration

Client:

- UI form
  - handler
  - useAuth()
- NavBar
  - useAuth()

Hook:
hooks/useAuth.tsx

- register
- isAuthenticated
- Set User state
- isLoading

Server routes

- Register
  - hashPassword
  - Prisma – find and add user

Conventional Custom Credentials

- npm i bcryptjs
- npm i jsonwebtoken
- npm i @types/jsonwebtoken
- npm i tailwindcss @tailwindcss/postcss daisyui@latest
- npm install prisma @prisma/client @prisma/adapter-pg pg
- .env - JWT_SECRET=

Generate JWT secret

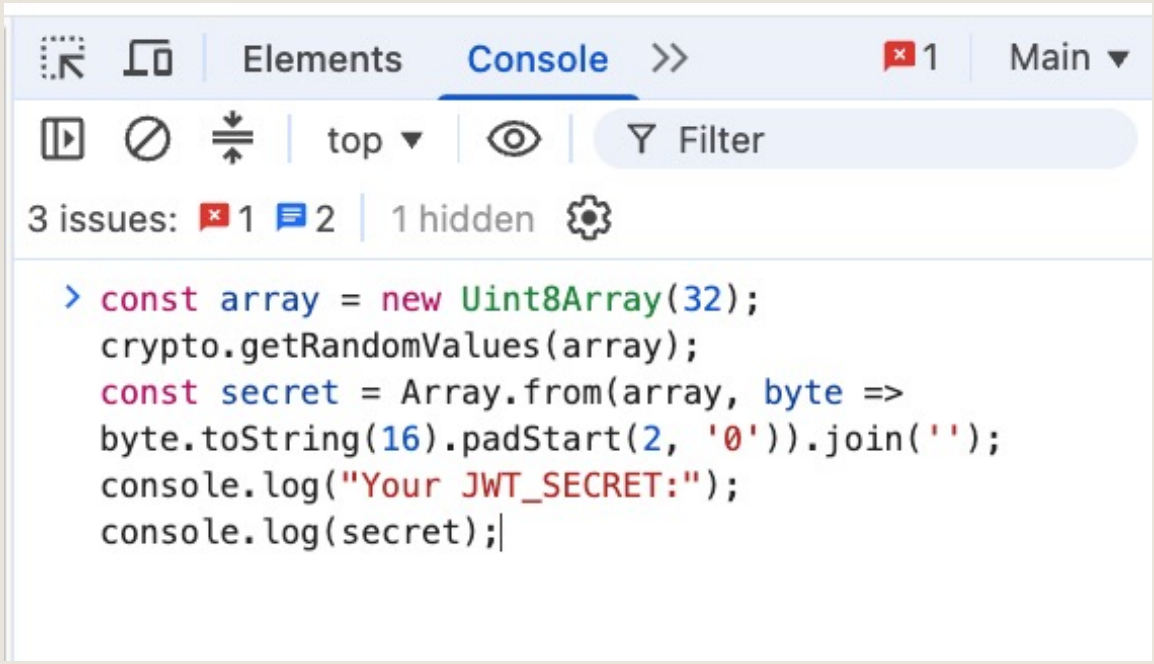
- node –e "console.log(require('crypto').randomBytes(32).toString('base64’))”

OR

- openssl rand -base64 32

OR

Using a web console:



Ge

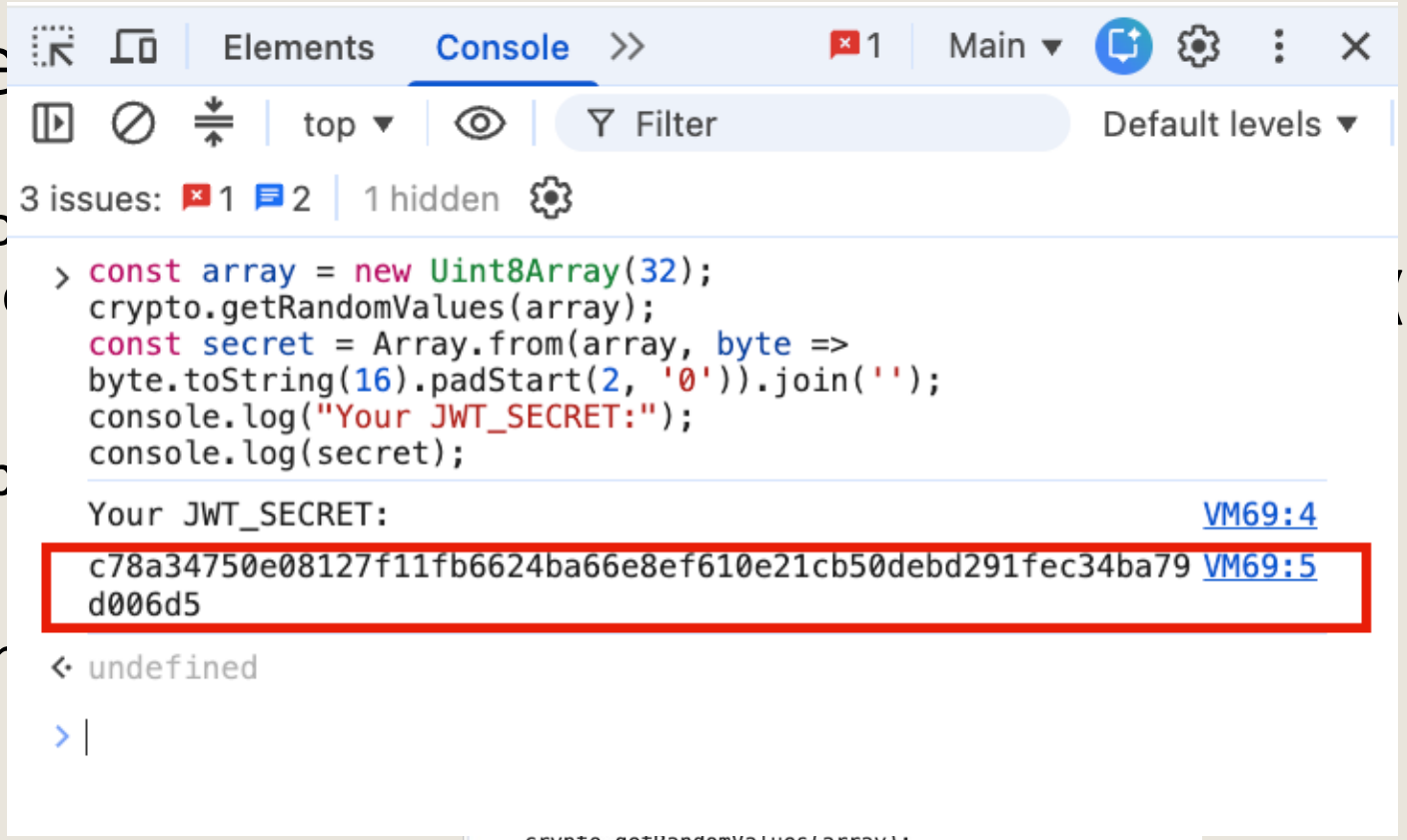
- nc
  "c ('base64'))"

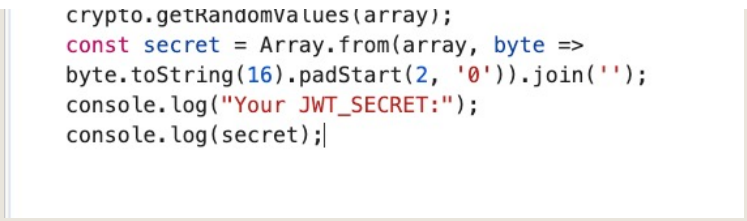
OR

- op

OR

Usir





Let’s start with server-side implementation

/api/auth/register/route.ts

Server Register API

Api/auth/register/route.ts

Post route

Verify request body

Check if the user already exists?

app > api > auth > register > TS route.ts > POST > existingUser > where

```ts
import { NextRequest, NextResponse } from "next/server";
import { prisma } from "../../../lib/prisma";
import { hashPassword, generateToken, setAuthCookie } from "../../../lib/auth";

export async function POST(request: NextRequest) {
  try {
    const body = await request.json();
    const { email, password, name } = body;

    // Validate input
    if (!email || !password || !name) {
      return NextResponse.json(
        { error: "Email, password, and name are required" },
        { status: 400 }
      );
    }

    // Check if user already exists
    const existingUser = await prisma.user.findUnique({
      where: { email }
    });

    if (existingUser) {
      return NextResponse.json(
        { error: "User with this email already exists" },
        { status: 409 }
      );
    }
```

Server
Register API

Api/auth/register/route.ts

Hash the password

Create the new user

Generate response

```
29
30    // Hash password and create user
31    const hashedPassword = await hashPassword(password);
32    const user = await prisma.user.create({
33      data: {
34        email,
35        password: hashedPassword,
36        name
37      }
38    });
```

```
47      // Create response with httpOnly cookie
48      const response = NextResponse.json(
49        {
50          message: "User registered successfully",
51          user: {
52            id: user.id,
53            email: user.email,
54            name: user.name
55          }
56        },
57        { status: 201 }
58      );
```

Server
Register API

Api/auth/register/route.ts

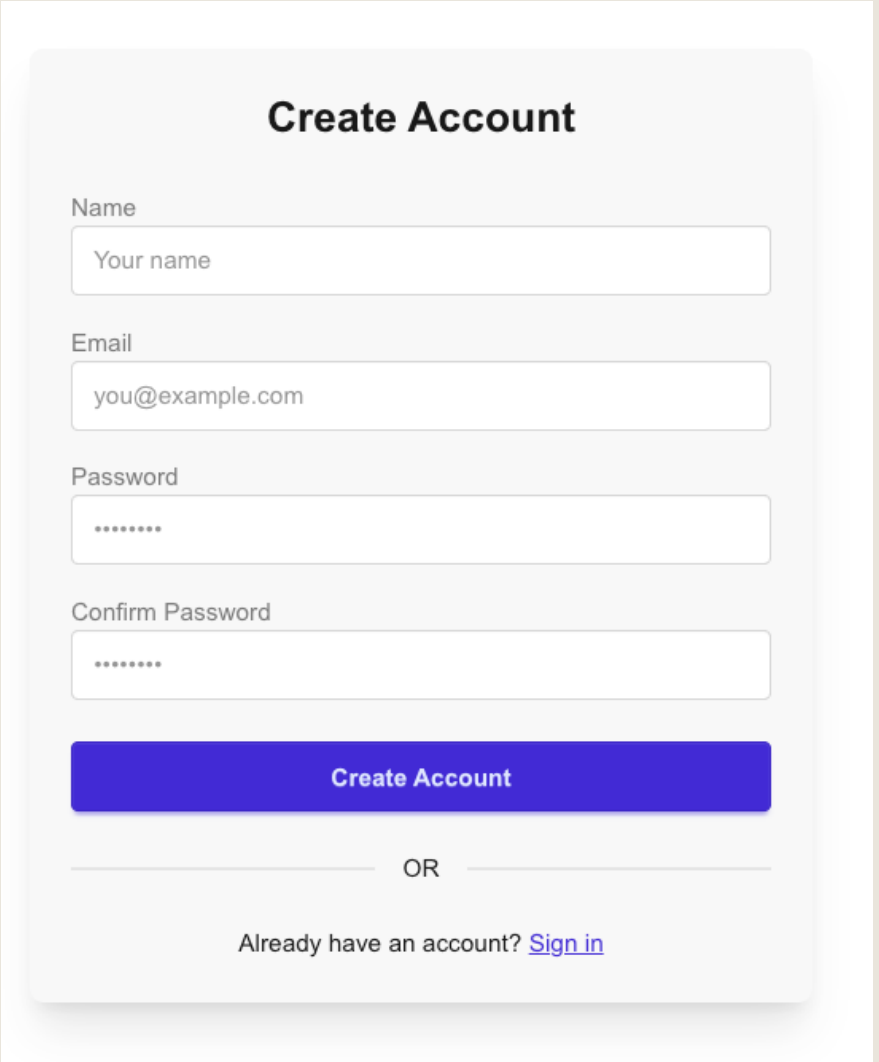
Return response

Handle errors

```
63      return response;
64    } catch (error) {
65      console.error("Registration error:", error);
66      return NextResponse.json(
67        { error: "Internal server error" },
68        { status: 500 }
69      );
70    }
71  }
```

Client-side implementation

UI & custom hook



Client Registration UI

app/(auth)/register/page.tsx

Custom hook – register, isAuthenticated, isLoading

app > (auth) > register > page.tsx > RegisterPage

```tsx
"use client";

import Link from "next/link";
import { useState, useEffect } from "react";
import { useAuth } from "../../hooks/useAuth";
import { useRouter } from "next/navigation";

const RegisterPage = () => {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");
  const [error, setError] = useState("");
  const [isSubmitting, setIsSubmitting] = useState(false);
  const { register, isAuthenticated, isLoading } = useAuth();
  const router = useRouter();
```

Client Registration UI

app/(auth)/register
/page.tsx

Redirect if already logged in

```
17
18    // SAFELY handle redirect if already logged in
19    useEffect(() => {
20      if (!isLoading && isAuthenticated) {
21        router.push("/");
22      }
23    }, [isLoading, isAuthenticated, router]);
24
```

Client Registration UI

app/(auth)/register
/page.tsx

Register the user

success go to /

```tsx
const handleSubmit = async (e: React.SubmitEvent<HTMLFormElement>) => {
  e.preventDefault();
  setError("");

  if (password !== confirmPassword) {
    setError("Passwords do not match");
    return;
  }

  if (password.length < 6) {
    setError("Password must be at least 6 characters");
    return;
  }

  setIsSubmitting(true);

  try {
    await register({ email, password, name });
    router.push("/login");
  } catch (err) {
    setError(err instanceof Error ? err.message : "Registration failed");
  } finally {
    setIsSubmitting(false);
  }
};
```

Client Registration UI

app/(auth)/register/page.tsx

Attach the handler to <form>



```
51    return (
52      <>
53        <h2 cla ame="card-ti e text-2xl justify-center">Create Account</h2>
54        <form onSubmit={handleSubmit} className="mt-4">
55          <div className="form-control">
56            <label className="label">
57              <span className="label-text">Name</span>
58            </label>
```

Client – recall custom hook useAuth()

```
21  interface User {
22    id: string | null | undefined;
23    email: string | null | undefined;
24    name: string | null | undefined;
25  }
26
27  interface AuthState {
28    user: User;
29    isAuthenticated: boolean;
30    isLoading: boolean;
31  }
```

Define global state user and functions for checking this user

Modify Custom hook – useAuth()

app/hook/useAuth.tsx

Add Login and Register credentials

Add register type in global state type

```
32
33    interface LoginCredentials {
34      email: string;
35      password: string;
36    }
37
38    interface RegisterCredentials extends LoginCredentials {
39      name: string;
40    }
41
```

```
41
42    interface AuthResponse {
43      user: User;
44      message?: string;
45      error?: string;
46    }
47
48    interface AuthContextType extends AuthState {
49      login: (credentials: LoginCredentials) => Promise<AuthResponse>;
50      register: (credentials: RegisterCredentials) => Promise<AuthResponse>;
51      checkGoogle: () => void;
52      logout: () => Promise<void>;
53    }
54
```

Modify Custom hook – useAuth()

app/hook/useAuth.tsx

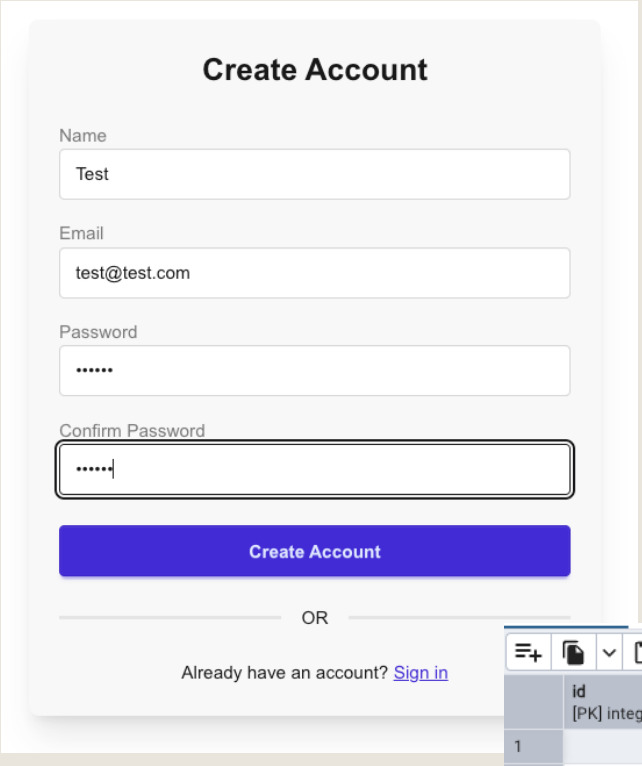
Register function

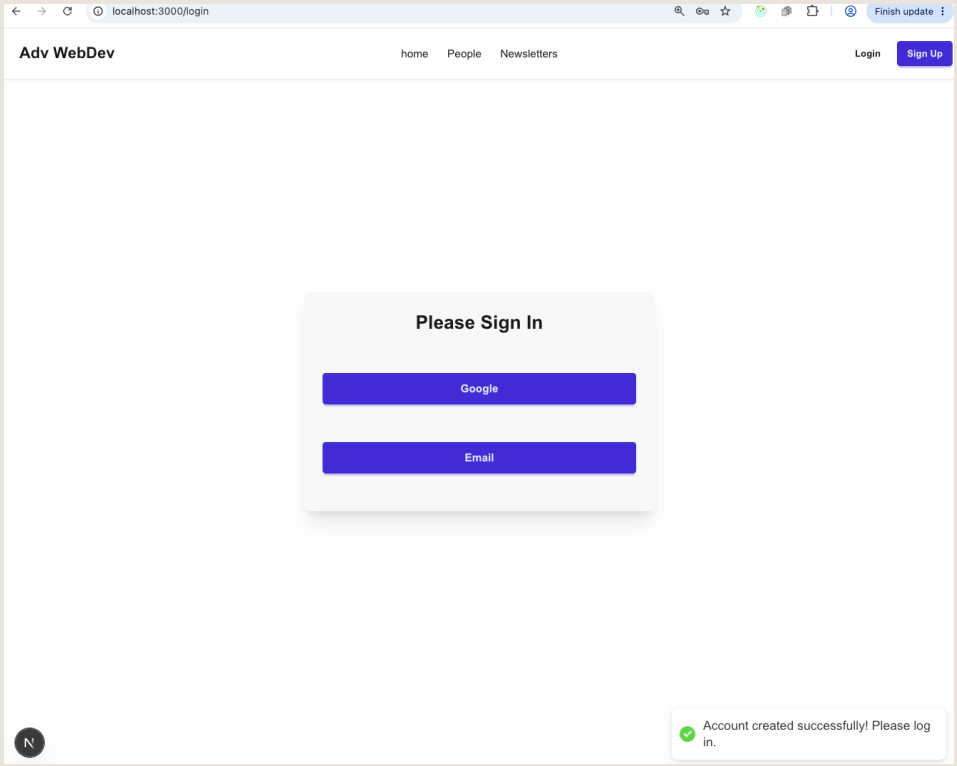
Add to the global state

```
165  const register = useCallback(
166    async (credentials: RegisterCredentials): Promise<AuthResponse> => {
167      const response = await fetch("/api/auth/register", {
168        method: "POST",
169        headers: { "Content-Type": "application/json" },
170        body: JSON.stringify(credentials),
171        credentials: "include"
172      });
173
174      const data: AuthResponse = await response.json();
175
176      if (!response.ok) {
177        throw new Error(data.error || "Registration failed");
178      }
179
180      return data;
181    },
182    []
183  );
```

```
189
190    const value: AuthContextType = {
191      ...authState,
192      login,
193      register,
194      checkGoogle,
195      logout
196    };
197
```

Test it!


Create Account
Name
Test
Email
test@test.com
Password
Confirm Password
Create Account
OR
Already have an account? Sign in


localhost:3000/login
Finish update
Adv WebDev
home
People
Newsletters
Login
Sign Up
Please Sign In
Google
Email
Account created successfully! Please log in.


Showing rows: 1 to 4
Page No: 1
of 1
id [PK] integer
email text
password text
name text
created_at timestamp without time zone
1 1 jc@jc.com $2b$10$q1F9hdCpveQV1AGz4iJkieEfeQBu7T3eOE3FH7VzH2xz9RyZsi... Jo Blog (Edited) (Edited) (Edit... 2026-07-24 16:16:33.472
2 3 user@test.c... 123456 Web User 2026-08-05 17:30:49.475
3 5 jane@test.c... 123456 Jane Smith 2026-10-04 13:23:41.437
4 8 test@test.co... $2b$10$jS9xDrKTV3p05VCL8NqyRufC1/DpW1tGZ9BH4UKzD1dESRl4z... Test 2026-10-18 14:13:47.602

Conventional Custom Credentials

Login and Logout

Manual responsibility for parsing input, database checks, password hashing, JWT signing, setting secure `HTTP-only` cookie headers, error handling etc.

Conventional Custom Credentials

Client:

- UI form
  - handler
  - useAuth()
- NavBar
  - useAuth()

Hook:
hooks/useAuth.tsx
- check session
- Set User state
- logout
- LocalStorage (set and remove)

Server routes

- Login
  - -prisma
  - -create JWT
- Logout
  - - remove JWT
- Me
  - - JWT verification

Server Routes

Server-side handling:

/api/auth/login/route.ts
/api/auth/logout/route.ts
/api/auth/me/route.ts

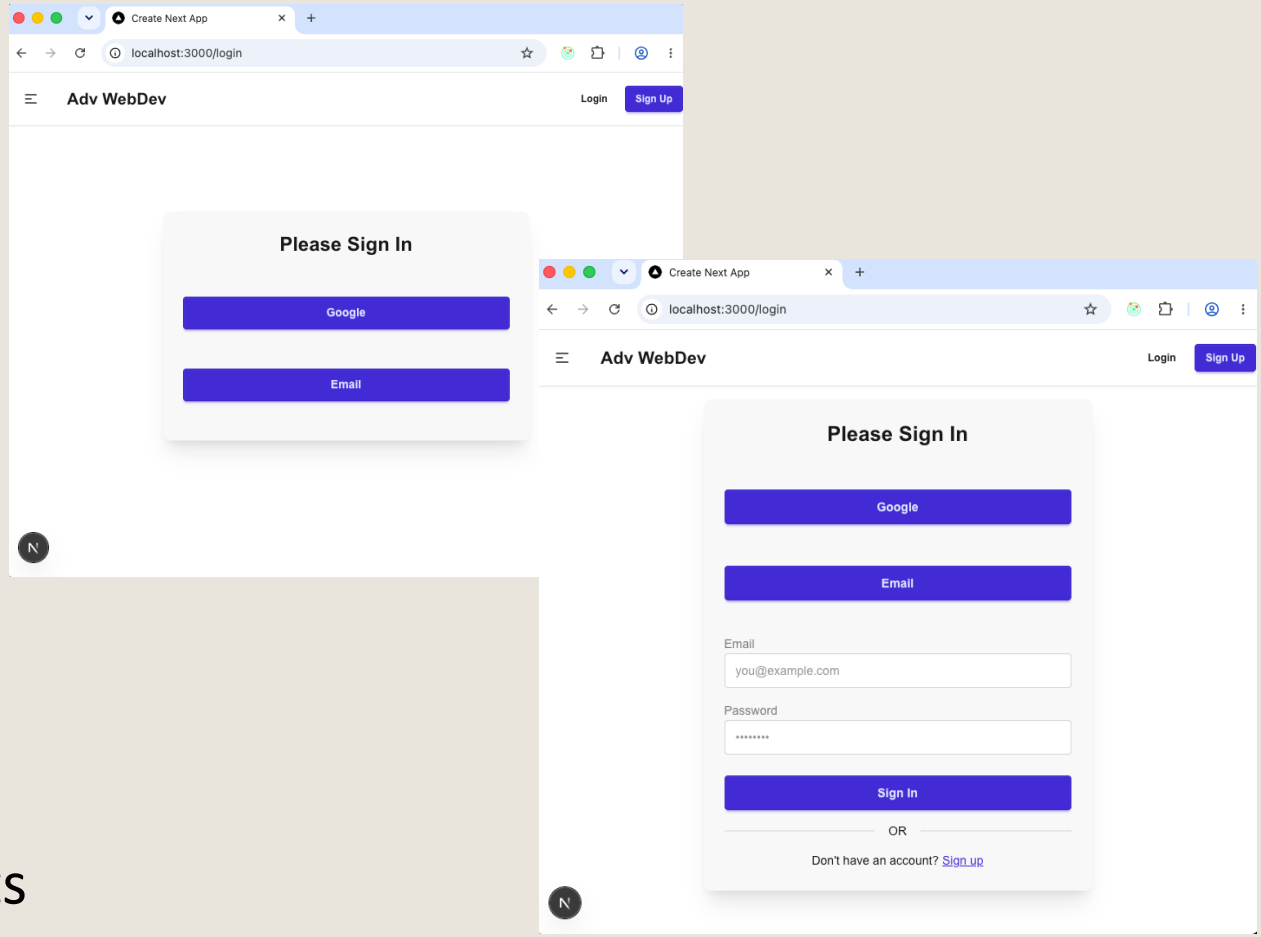
Client & Proxy

UI

- Form
- NavBar

Hook – useAuth

Proxy – securing end points



Let’s start with server-side implementation

/api/auth/login/route.ts

Import libraries

app > api > auth > login > TS route.ts > POST

```
1  import { NextResponse } from "next/server";
2  import bcrypt from "bcryptjs";
3  import jwt from "jsonwebtoken";
4  import { prisma } from "../../../lib/prisma";
5
```

/api/auth/login/route.ts

Form submit – POST route

Basic check

Find the user

```
6   export async function POST(req: Request) {
7     try {
8       const { email, password } = await req.json();
9
10      // 1. Manual input validation
11      if (!email || !password) {
12        return NextResponse.json(
13          { error: "Missing required fields" },
14          { status: 400 }
15        );
16      }
17
18      // 2. Fetch user from database
19      const user = await prisma.user.findUnique({ where: { email } });
20      if (!user) {
21        return NextResponse.json(
22          { error: "Invalid credentials supplied" },
23          { status: 401 }
24        );
25      }
26
```

/api/auth/login/route.ts

Once the user is found:

Check the credentials

```
27    // 3. Verify password hash manually
28    const isValidPassword = await bcrypt.compare(password, user.password);
29    if (!isValidPassword) {
30      return NextResponse.json(
31        { error: "Invalid credentials supplied" },
32        { status: 401 }
33      );
34    }
35
```

/api/auth/login/route.ts

Valid Password :
Grab secret from .env

Generate token

Generate Response

Set Session cookie

HttpOnly

```
35
36  // Guard check for secret
37  const secret = process.env.JWT_SECRET;
38  if (!secret) {
39    throw new Error("JWT_SECRET environment variable is not defined");
40  }
41  // 4. Manually sign JWT session token
42  const token = jwt.sign(
43    { userId: user.id, email: user.email, name: user.name },
44    secret,
45    {
46      expiresIn: "7d"
47    }
48  );
```

```
46  // 5. Build response and manually write security cookie headers
47  const response = NextResponse.json({
48    success: true,
49    user: { id: user.id, email: user.email }
50  });
51
52  response.cookies.set({
53    name: "custom_session_token",
54    value: token,
55    httpOnly: true, // Protects from XSS read access
56    secure: process.env.NODE_ENV === "production",
57    sameSite: "lax", // Protects against CSRF
58    path: "/",
59    maxAge: 60 * 60 * 24 * 7 // 7 days
60  });
61
62  return response;
```

Client-side implementation

Client - Custom hook useAuth()

```
21  interface User {
22    id: string | null | undefined;
23    email: string | null | undefined;
24    name: string | null | undefined;
25    provider?: "conventional credentials" | string; // track login provider
26  }
```

```
48
49  interface AuthContextType extends AuthState {
50    login: (credentials: LoginCredentials) => Promise<AuthResponse>;
51    register: (credentials: RegisterCredentials) => Promise<AuthResponse>;
52    checkGoogle: () => void;
53    logout: () => Promise<void>;
54  }
```

Define global state user and functions for checking this user

Client - Custom hook useAuth()

```
93
94  const login = useCallback(
95    async (credentials: LoginCredentials): Promise<AuthResponse> => {
96      const response = await fetch("/api/auth/login", {
97        method: "POST",
98        headers: { "Content-Type": "application/json" },
99        body: JSON.stringify(credentials),
100       credentials: "include"
101     });
102
103     const data: AuthResponse = await response.json();
104
105     if (!response.ok) {
106       throw new Error(data.error || "Login failed");
107     }
108
109     // Update global state immediately
110     setAuthState({
111       user: data.user,
112       isAuthenticated: true,
113       isLoading: false
114     });
115
116     return data;
117   },
118   []
119 );
120
```

Client - Custom hook useAuth()

Check both custom and Google Sessions

useCallback hook:

https://react.dev/reference/react/useCallback

```
const checkSession = useCallback(async () => {
  try {
    // 1. Verify Custom JWT Cookie against the server endpoint
    const res = await fetch("/api/auth/me");
    if (res.ok) {
      const data = await res.json();
      const userData = {
        id: data.user.id,
        email: data.user.email,
        name: data.user.name || data.user.email,
        provider: "conventional credentials"
      };

      setAuthState({
        user: userData,
        isAuthenticated: true,
        isLoading: false
      });
      return; // Stop here: Custom credentials session verified!
    }

    // 2. Fallback: Check Google Session
    const googleSession = await checkGoogleSession();

    if (googleSession?.user) {
      const userData = {
        id: googleSession.user.id,
        name: googleSession.user.name,
        email: googleSession.user.email,
        provider: "google"
      };
      // Save to localStorage
      localStorage.setItem("custom_user", JSON.stringify(userData));
      setAuthState({
        user: userData,
        isAuthenticated: true,
        isLoading: false
      });
      return;
    }

    // No active session
    setAuthState({
      user: emptyUser,
      isAuthenticated: false,
      isLoading: false
    });
  } catch (error) {
    setAuthState({
      user: emptyUser,
      isAuthenticated: false,
      isLoading: false
    });
  }
}, []);

// Check auth status on mount
useEffect(() => {
  checkSession();
}, [checkSession]);
```

The Importance of Session Verification

Session verification

- Local Storage or dedicated end point?
- Never store JWT in Local Storage !
- If an application stores the secret session JWT in `localStorage` and reads it via JavaScript to attach to request headers :



Warning Warning Warning

- **XSS Vulnerability:** Any JavaScript code running on your page — including third-party analytics, npm packages, or malicious scripts injected via Cross-Site Scripting (XSS)— can execute `localStorage.getItem("jwt")` and steal the user's active session token.

- **No Server Defence:** Once an attacker reads a JWT out of `localStorage`, they can exfiltrate it to their own server and impersonate the user from anywhere in the world until the token expires.

Local Storage

Safe & Standard Client Cache

Store **non-sensitive user profile metadata** in `localStorage` is OK

e.g. { name: "Alice", email: "alice@example.com" }

Cookie Approach + UI Cache Is Safe



Secret JWT is stored in an `HttpOnly` cookie.

- JavaScript running in the browser **cannot read this cookie**. Even if your app suffers an XSS vulnerability, the attacker cannot steal the raw JWT token out of the cookie.
- Browsers send `HttpOnly` cookies automatically on every request to /api/*.

Cookie Approach + UI Cache Is Safe



`localStorage` only stores cosmetic UI data.

- Storing { id: "123", email: "user@email.com", name: "Alice" } in `localStorage` is completely safe because that object contains **no credentials, secrets, or signature keys**.
- It is purely used to prevent the `Navbar` from flickering while React mounts.

Session Verification Options

Dimension	Option A: Server Verification (/api/auth/me)	Option B: Client UI Cache (localStorage)
Security	**Highest.** The server validates and verifies the `HttpOnly` JWT on every page refresh.	**Medium.** Safe from XSS token theft, but if the cookie expires on the backend, the UI might briefly show "Logged In" until an API call fails.
Performance	**1 Extra Network Request** on every page refresh.	**Instant.** Synchronous read from local storage with 0ms network latency.
Implementation Complexity	Requires creating and maintaining a JWT verification API endpoint.	Requires zero new endpoints.

Option A: Server-side verification

/api/auth/me/route.ts

app > api > auth > me > TS route.ts > ...

```ts
import { NextResponse } from "next/server";
import { cookies } from "next/headers";
import jwt from "jsonwebtoken";

export async function GET() {
  try {
    // 1. Extract the HTTP-only cookie
    const cookieStore = await cookies();
    const token = cookieStore.get("custom_session_token")?.value;

    if (!token) {
      return NextResponse.json({ error: "Not authenticated" }, { status: 401 });
    }

    // 2. Verify the JWT signature & expiration
    const secret = process.env.JWT_SECRET;
    if (!secret) {
      throw new Error("JWT_SECRET is missing");
    }

    const decoded = jwt.verify(token, secret) as {
      userId: string;
      email: string;
    };

    // 3. Return verified user payload
    return NextResponse.json({
      success: true,
      user: {
        id: decoded.userId,
        email: decoded.email,
        name: decoded.email // Or fetch fresh user from DB if needed
      }
    });
  } catch (error) {
    // JWT signature invalid or expired
    return NextResponse.json({ error: "Invalid token" }, { status: 401 });
  }
}
```

Continue Client - Custom hook `useAuth()`

Clean up on logout

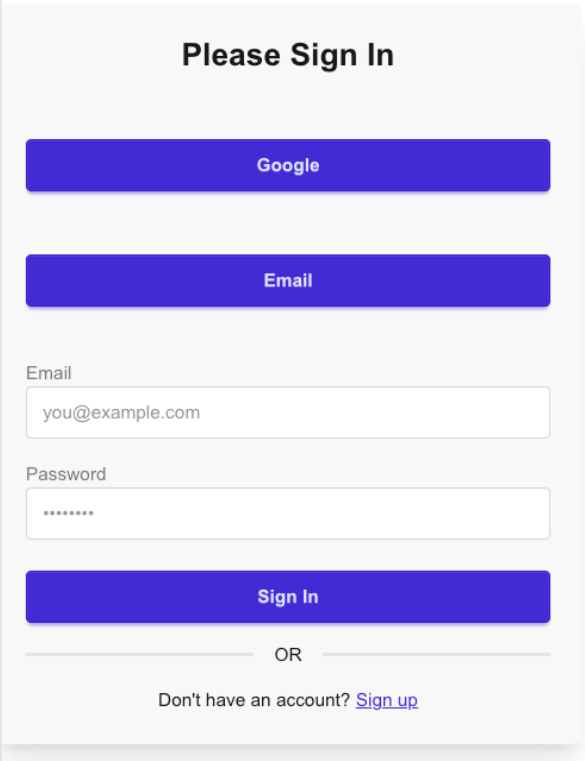
```
142
143   const logout = useCallback(async () => {
144     try {
145       // 1. Clear Custom Credentials JWT Cookie on the server
146       await fetch("/api/auth/logout", { method: "POST" });
147
148       // 2. Clear Google Session (if Google signout action exists)
149       await handleGoogleSignout();
150     } catch (error) {
151       console.error("Logout error:", error);
152     } finally {
153       // 3. CLEAR LOCAL STORAGE
154       // Option A: Remove specific auth keys (Recommended)
155       localStorage.removeItem("custom_user");
156
157       // 4. Reset React State
158       setAuthState({
159         user: emptyUser,
160         isAuthenticated: false,
161         isLoading: false
162       });
163     }
```

Client Login Form

app > (auth) > login > CustomLoginForm.tsx > CustomLoginForm >

```tsx
"use client";
import Link from "next/link";
import { useState } from "react";
import { useAuth } from "../../hooks/useAuth";
import { useRouter } from "next/navigation";

const CustomLoginForm = () => {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const [isSubmitting, setIsSubmitting] = useState(false);
  const router = useRouter();
  const { setCustomAuthUser } = useAuth(); // Must expose a state updater in your AuthProvider
```



Client Login Form

Form Submit event handler

Request sends to end point

If successful, set global state

Refresh and Redirect

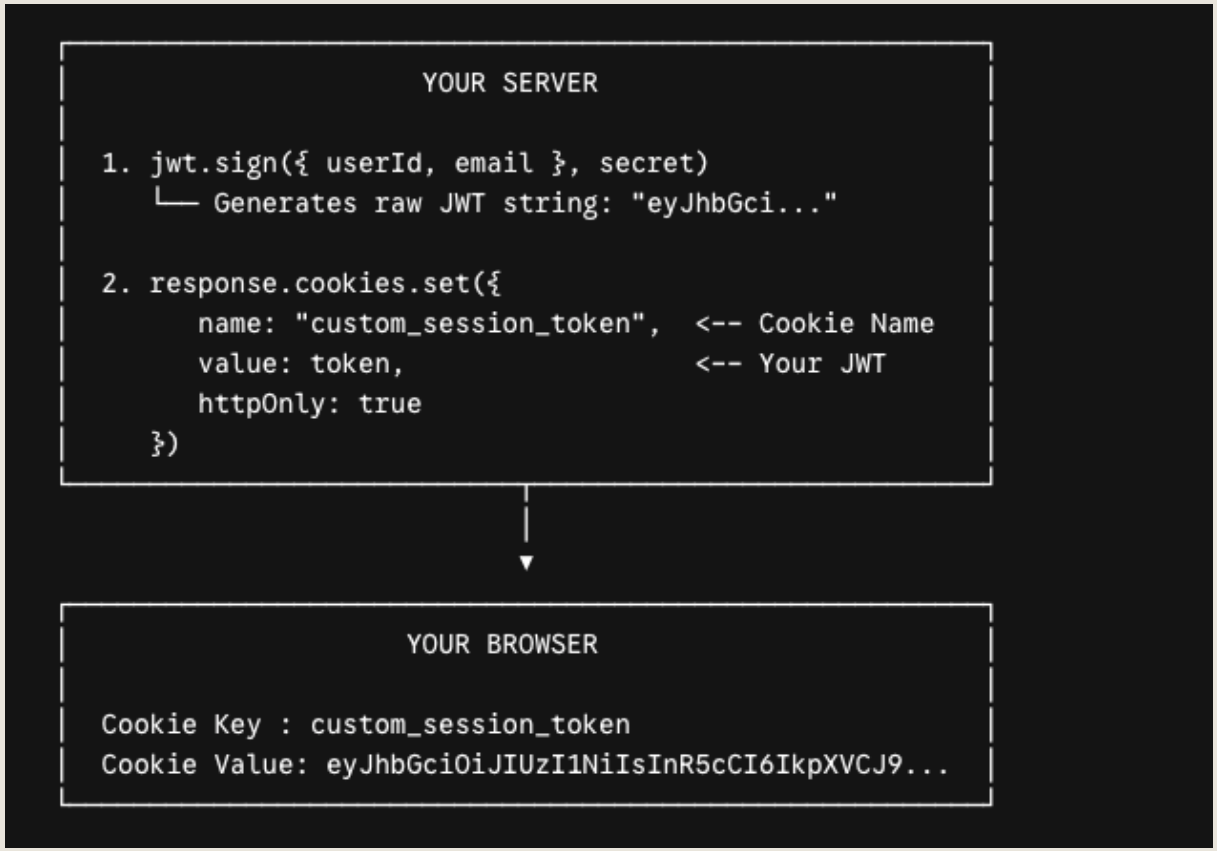
Without refreshing-
Next.js will often serve the page exactly as it was from the local cache rather than asking the server for new data.

```
const handleCustomSubmit = async (e: React.SubmitEvent<HTMLFormElement>) => {
  e.preventDefault();
  setError("");
  setIsSubmitting(true);
  try {
    // send the request to api for custom login
    const res = await fetch("/api/auth/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ email: email, password: password })
    });

    const data = await res.json();
    if (!res.ok) {
      setError(data.error || "An unexpected error occurred");
      return;
    }
    // Handle success (200 OK)
    // Immediately update React Auth Context state!
    // This forces the Navbar to re-render synchronously with user data.
    setCustomAuthUser({
      id: data.user.id,
      email: data.user.email,
      name: data.user.name || data.user.email
    });

    router.refresh();
    router.push("/");
  } catch (err) {
    setError(err instanceof Error ? err.message : "Login failed");
    router.push("/login");
  } finally {
    setIsSubmitting(false);
  }
};
```

What is happening under the hood



Proxy – Middleware for securing the end points

Check if there is current user

Public path (login and register)

Redirect appropriately

```
export function proxy(req: NextRequest) {
  const { pathname } = req.nextUrl;

  // 1. Extract the Approach 1 custom cookie
  const customToken = req.cookies.get("custom_session_token")?.value;
  const hasCustomAuth =
    typeof customToken === "string" && customToken.trim().length > 0;

  // 2. Check for Approach 2 & 3: Auth.js / NextAuth Session Cookies (Google & Auth.js Credentials)
  const authJsToken =
    req.cookies.get("authjs.session-token")?.value ||
    req.cookies.get("__Secure-authjs.session-token")?.value ||
    req.cookies.get("next-auth.session-token")?.value ||
    req.cookies.get("__Secure-next-auth.session-token")?.value;

  const hasAuthJsAuth =
    typeof authJsToken === "string" && authJsToken.trim().length > 0;

  // Ensure token exists AND is a non-empty string
  // const isAuthenticated =
  //   typeof customToken === "string" && customToken.trim().length > 0;

  // Combined Auth Check: User is logged in if ANY valid token exists
  const isAuthenticated = hasCustomAuth || hasAuthJsAuth;

  // 2. Define public routes that anyone can visit (Login and Register)
  const isPublicRoute =
    pathname.startsWith("/login") || pathname.startsWith("/register");

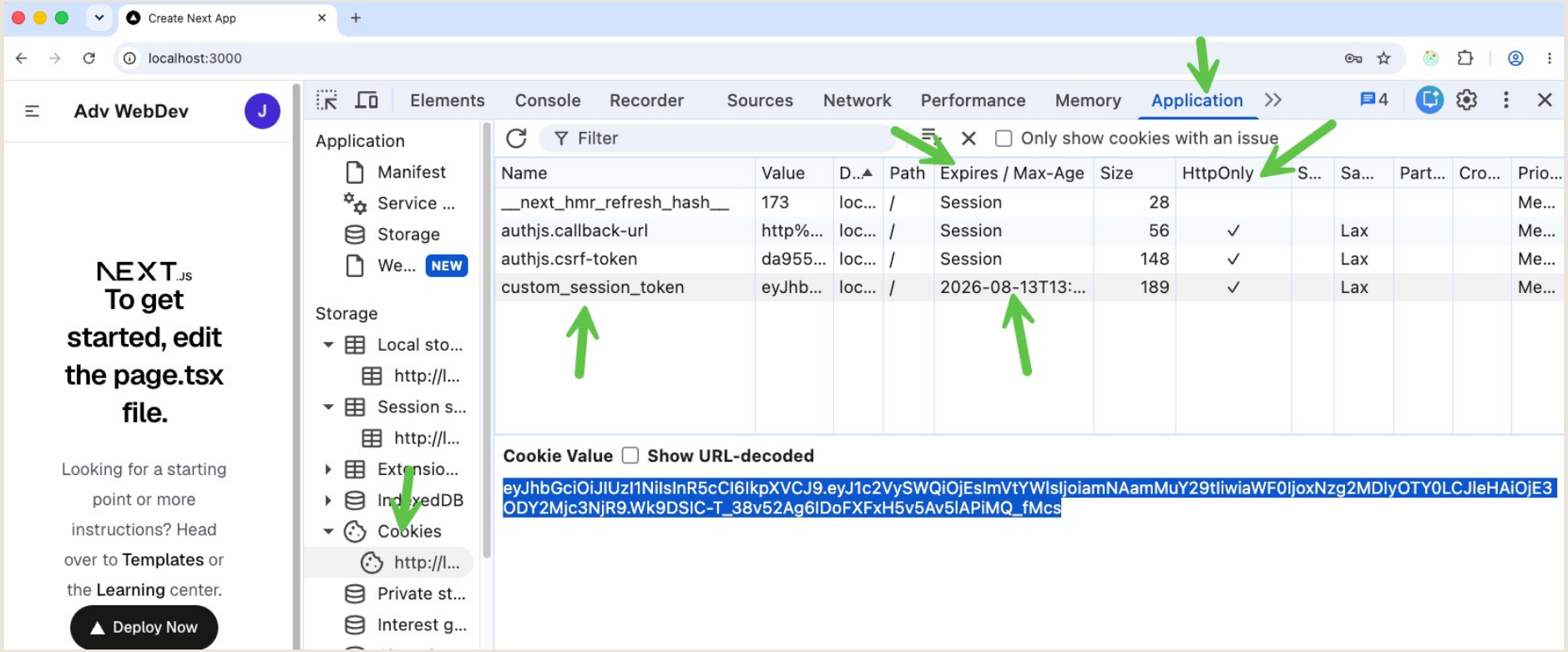
  // console.log(isAuthenticated);
  // console.log(isPublicRoute);

  // 3. UNAUTHENTICATED USER trying to access protected page -> Bounce to /login
  if (!isAuthenticated && !isPublicRoute) {
    return NextResponse.redirect(new URL("/login", req.nextUrl.origin));
  }

  // 4. AUTHENTICATED USER trying to visit /login or /register -> Bounce to home /
  if (isAuthenticated && isPublicRoute) {
    return NextResponse.redirect(new URL("/", req.nextUrl.origin));
  }

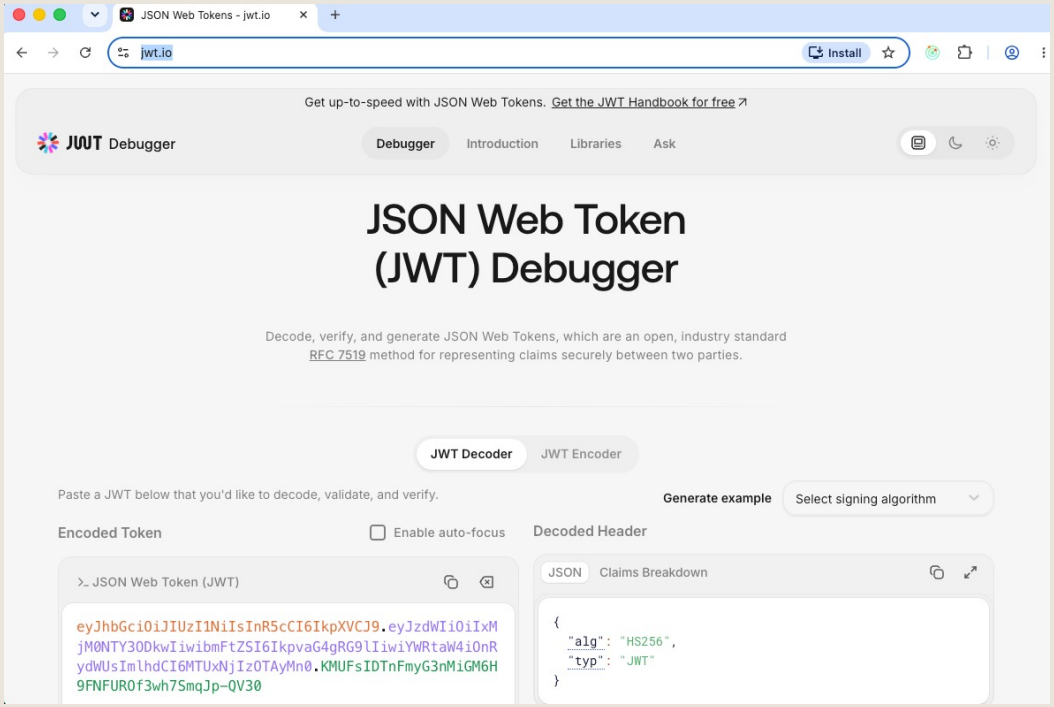
  return NextResponse.next();
}
```

Check out on Session Cookies



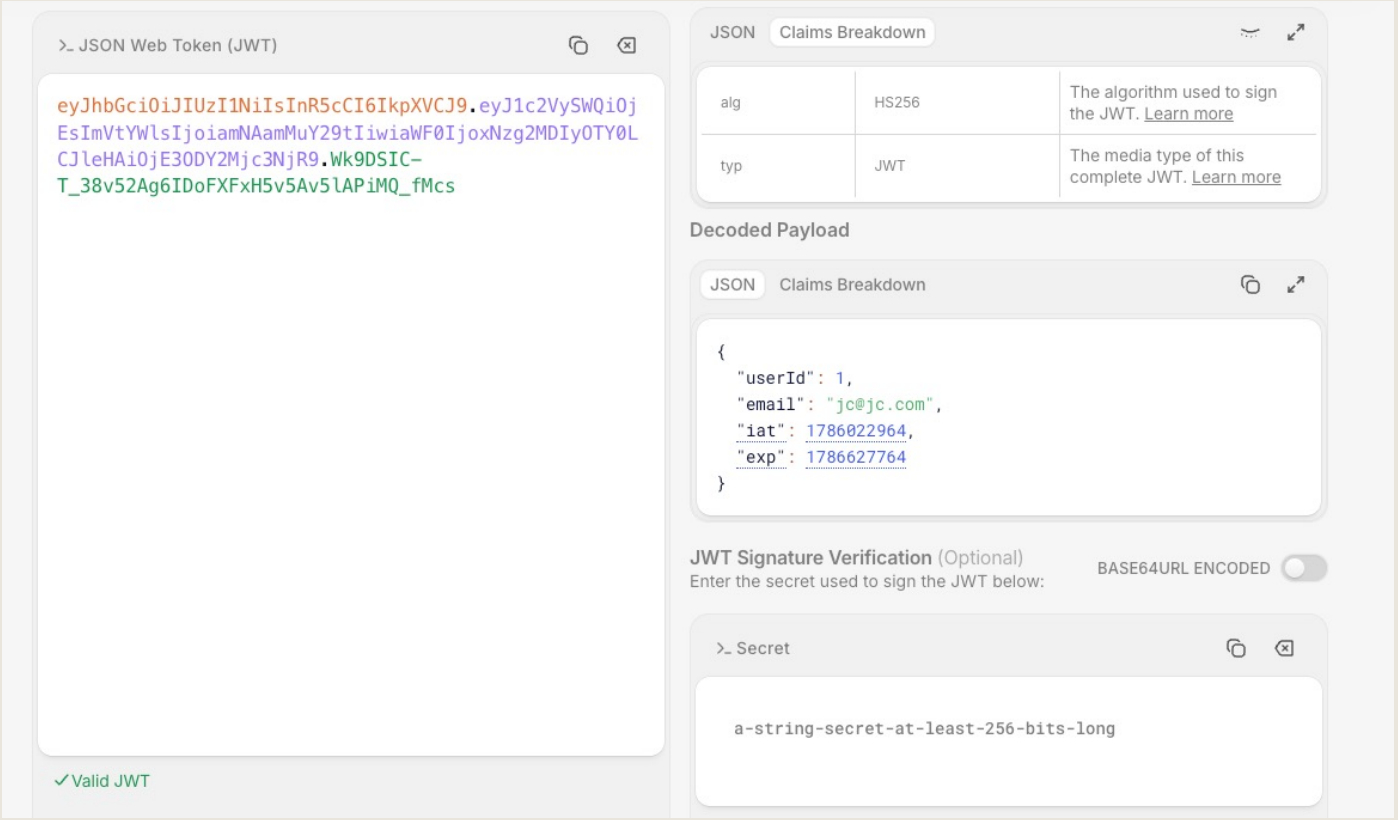
Inspect the JWT token

- Copy the token and paste it to https://www.jwt.io/



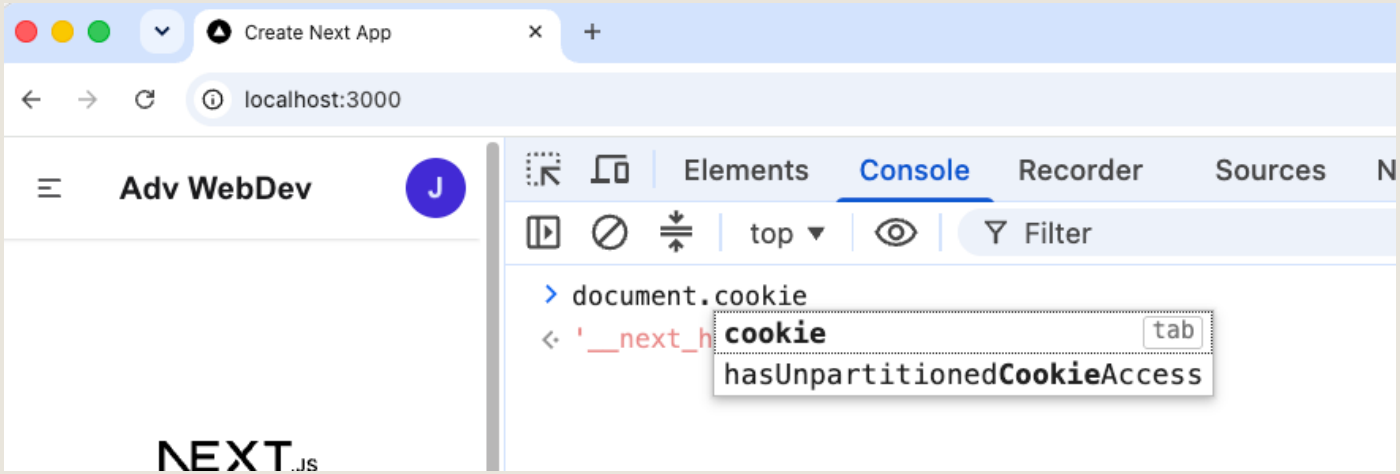
JWT token has 3 parts

- Header –
  - Defines the hashing algorithm
- Payload
  - Contains your manual claims
- Signature
  - Created using your `process.env.JWT_SECRET` so the server can verify the token hasn't been tampered with.



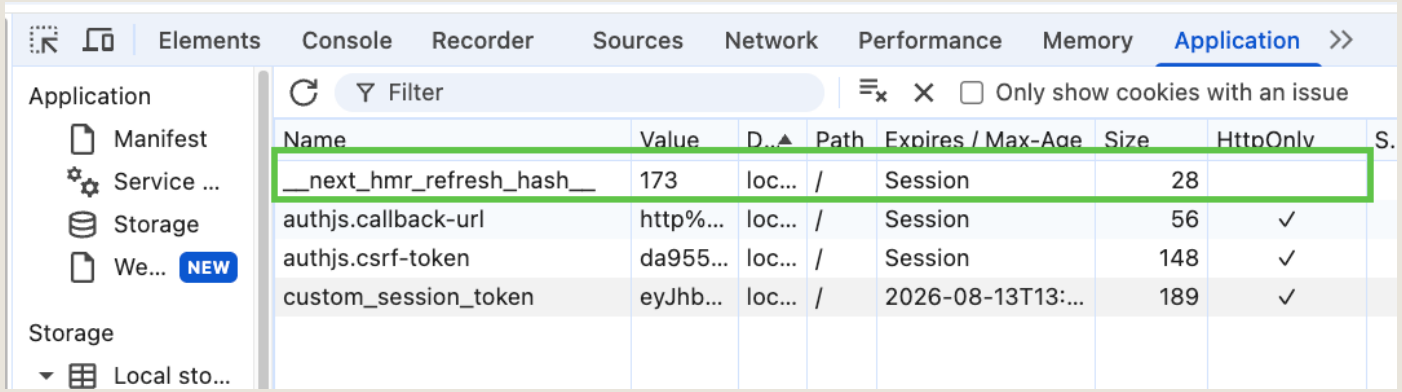
To prove JS cannot read HTTP-only cookie

- Go to console tab and type – document.cookie



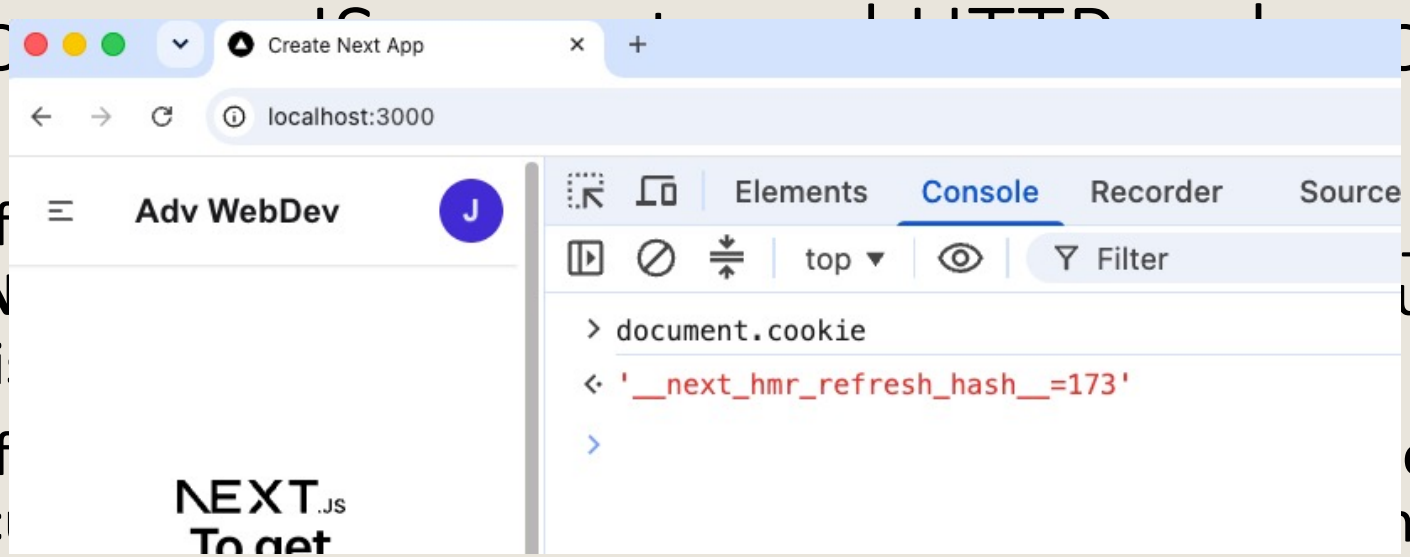
To prove JS cannot read HTTP-only cookie

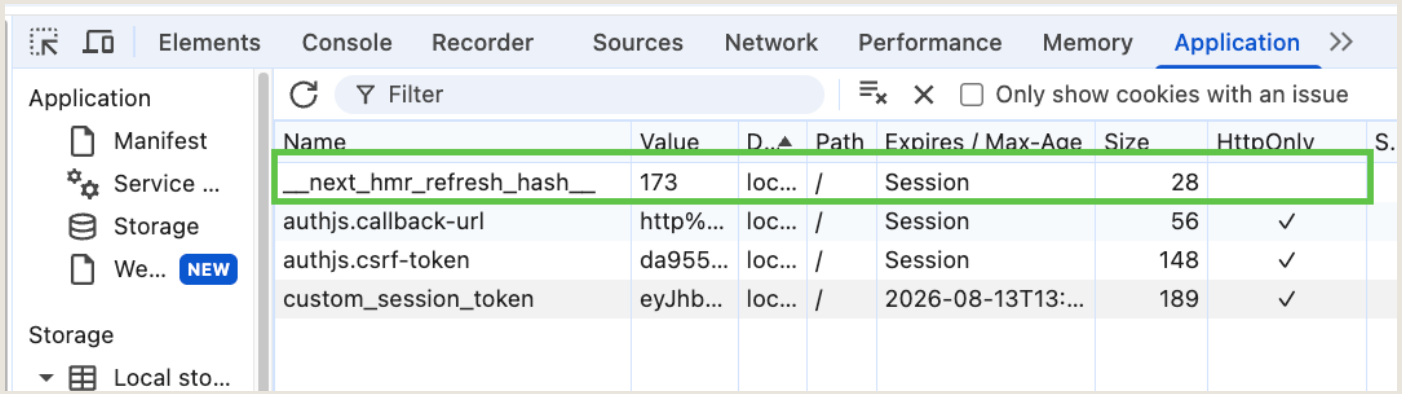
- If the cookie is **`HttpOnly`** (Protected) → `custom_session_token` will **NOT** appear in the output string at all! (Even though you can see it listed in the Application table).
- If the cookie is **Not HttpOnly** (Unprotected) →You will see custom_session_token=????? ... printed directly in the console.



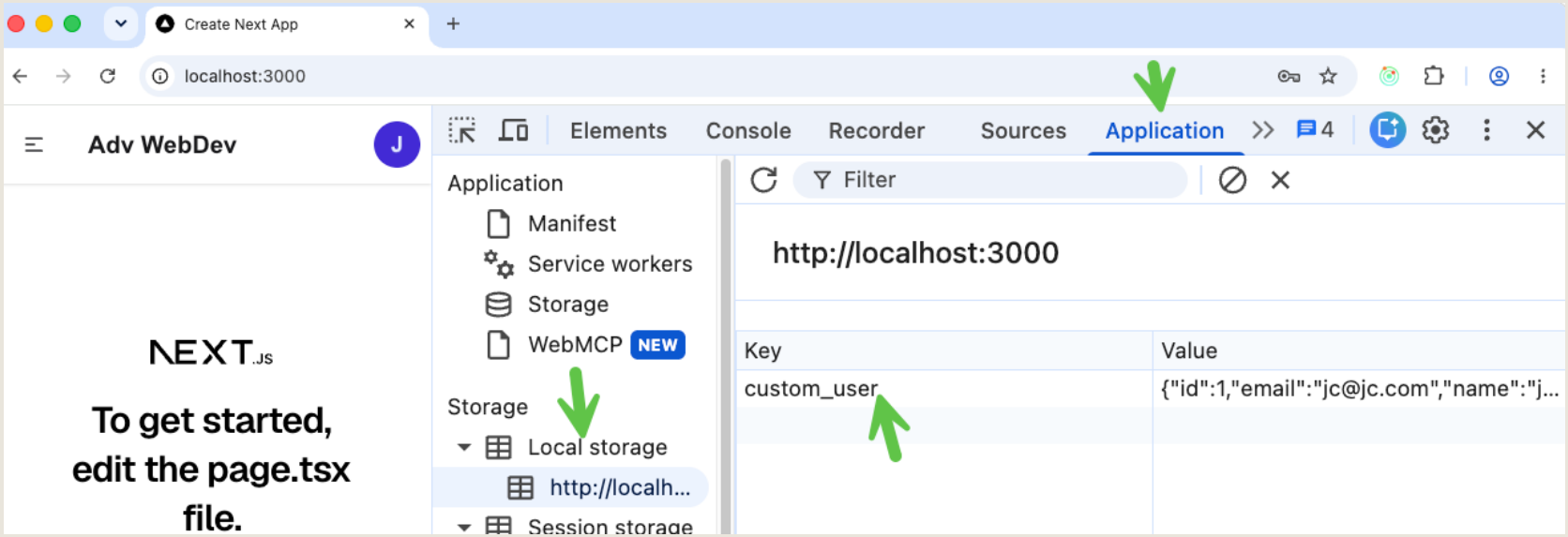
To ookie

- If _token will
  N u can see it
  li
- If ee
  c ne console.

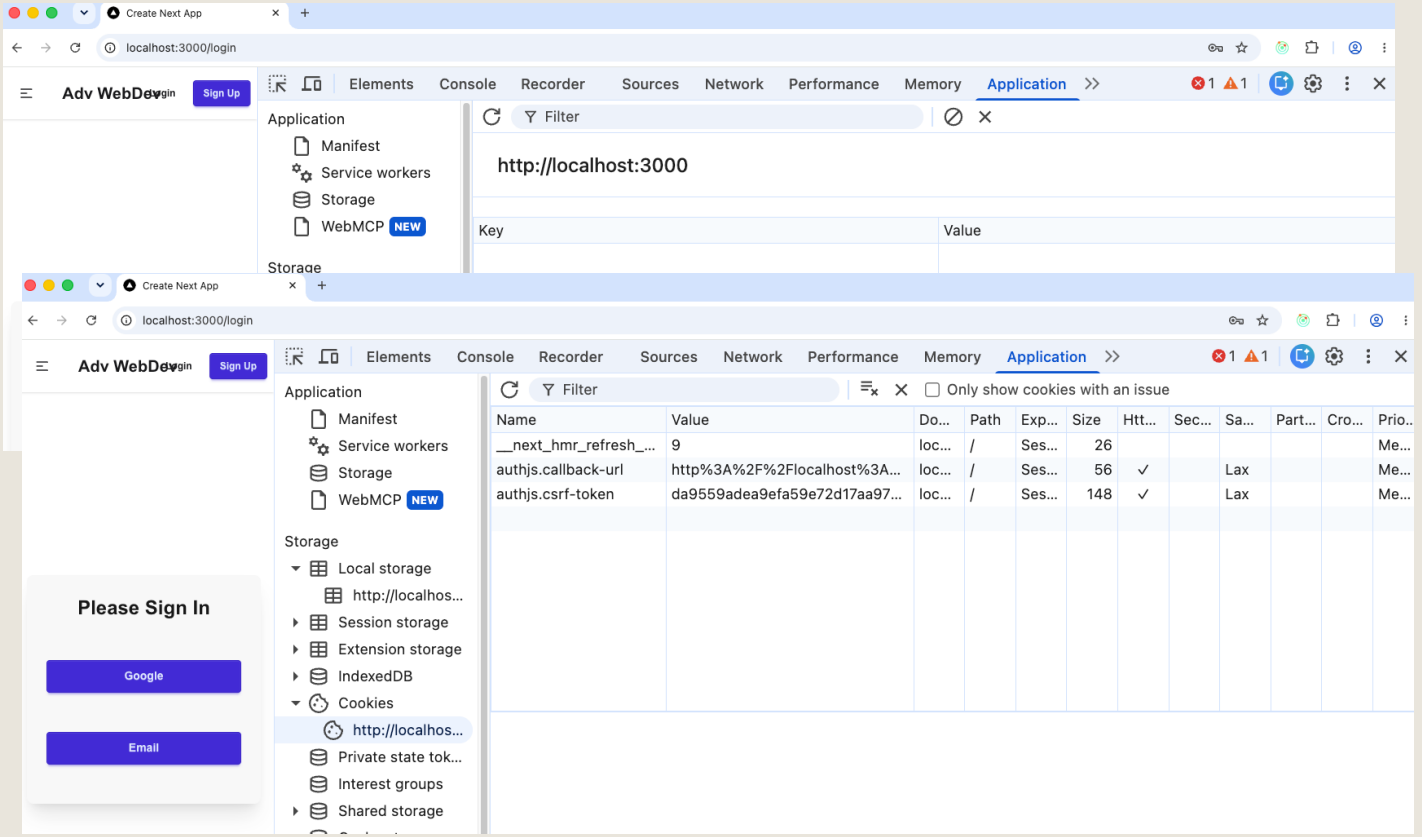




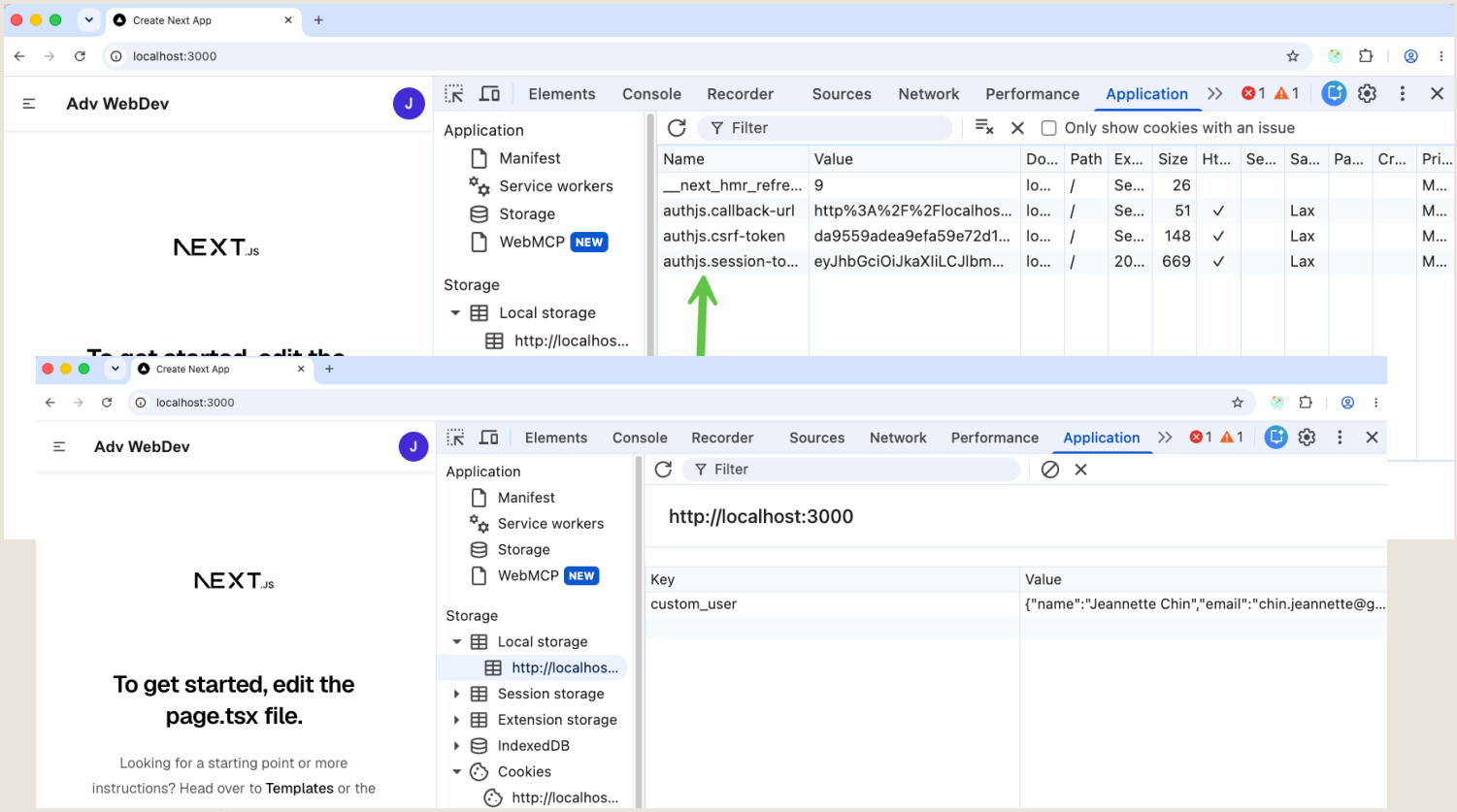
Check out the LocalStorage



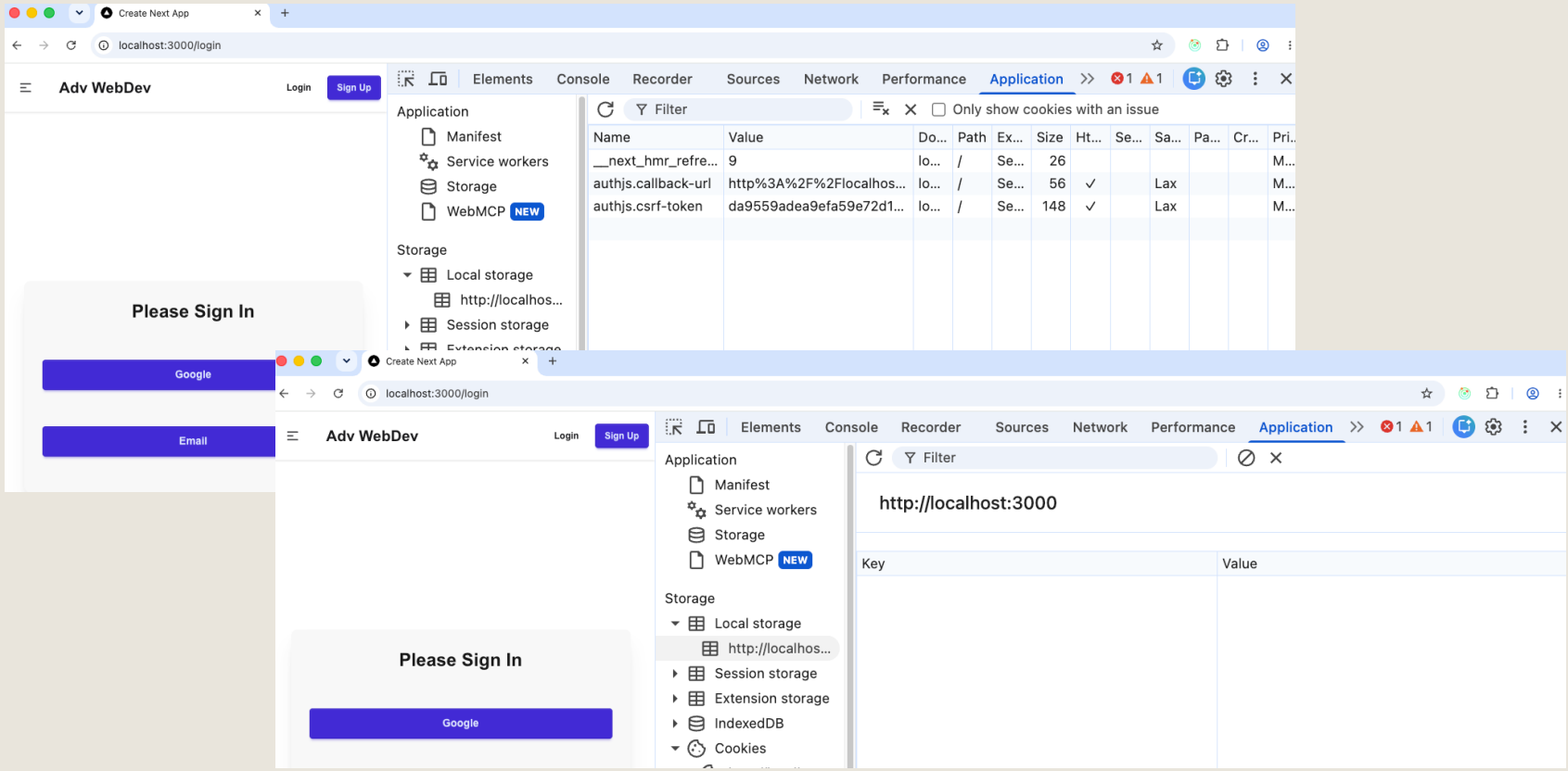
Clean up on logout evidence



Try Google Provider – on login



Try Google Provider – on logout



Cookies are the HTTP Ground Truth

Whenever your browser makes a request to your Next.js server (or middleware), the browser **automatically attaches HTTP cookies** to the request header.

- **The Server/Middleware does NOT look at React Context state** (useAuth, AuthProvider).
- **The Server/Middleware does NOT look at `localStorage`.**
- The server **only** sees the raw HTTP cookies sent along with the request header.
- If a valid token exists inside the incoming cookie header, the server considers the request **authenticated**. If no token exists, it considers the request **unauthenticated**.

React Context & `LocalStorage` are Secondary Caches

The `AuthProvider` (useAuth) and `localStorage` are just **client-side UI mirrors**. They exist solely so the React components (like Navbar or Dashboard) can render user details quickly without having to perform a full server fetch on every button click.

- **React Context (useAuth)** →UI State (re-renders buttons, shows user name etc).
- **LocalStorage** →Client persistence cache across page refreshes.
- **HTTP-Only Cookies** →**The Ground Truth** for security, middleware, and backend API routes.

The Golden Rule

- **If the cookie says you are logged out, you are logged out - no matter what React Context or `localStorage` claims.**

- Therefore, whenever a user logs in, logs out, or reloads the page, your app must align its client-side state (useAuth / localStorage) to match the **Ground Truth** of the cookie!

QR

Credential Provider – Auth.js

Approach 2

Auth.js CredentialsProvider

Auth.js takes over the session infrastructure. You only provide the user lookup & password verification logic (authorize()), and Auth.js manages the JWTs, cookies, session hooks, and middleware out of the box!

However, because I want the app to work with multiple approaches, the implementation is still the same!

Configure Auth.js

TS auth.ts > ...

```ts
import NextAuth from "next-auth";
import Google from "next-auth/providers/google";
import Credentials from "next-auth/providers/credentials";
import bcrypt from "bcryptjs";
import { prisma } from "./app/lib/prisma";

```

Add `bcrypt` and `Prisma` for database lookup + interactions

Auth.js

Set session strategy to JWT

code the authorize() with custom login

```
7   export const { handlers, signIn, signOut, auth } = NextAuth({
8     session: { strategy: "jwt" }, // Auth.js forces JWT for Credentials
9     providers: [
10      Google,
11      Credentials({
12        name: "Credentials",
13        credentials: {
14          email: { label: "Email", type: "email" },
15          password: { label: "Password", type: "password" }
16        },
17        // We ONLY write credential verification logic here
18        async authorize(credentials) {
19          if (!credentials?.email || !credentials?.password) return null;
20
21          const user = await prisma.user.findUnique({
22            where: { email: credentials.email as string }
23          });
24
25          if (!user) return null;
26
27          const isValid = await bcrypt.compare(
28            credentials.password as string,
29            user.password
30          );
31
32          if (!isValid) return null;
33
34          // Returning user object signals success; Auth.js handles JWT & cookies automatically
35          return {
36            id: String(user.id),
37            email: user.email,
38            name: user.name
39          };
40        }
41      })
42    ],
43    callbacks: {
```

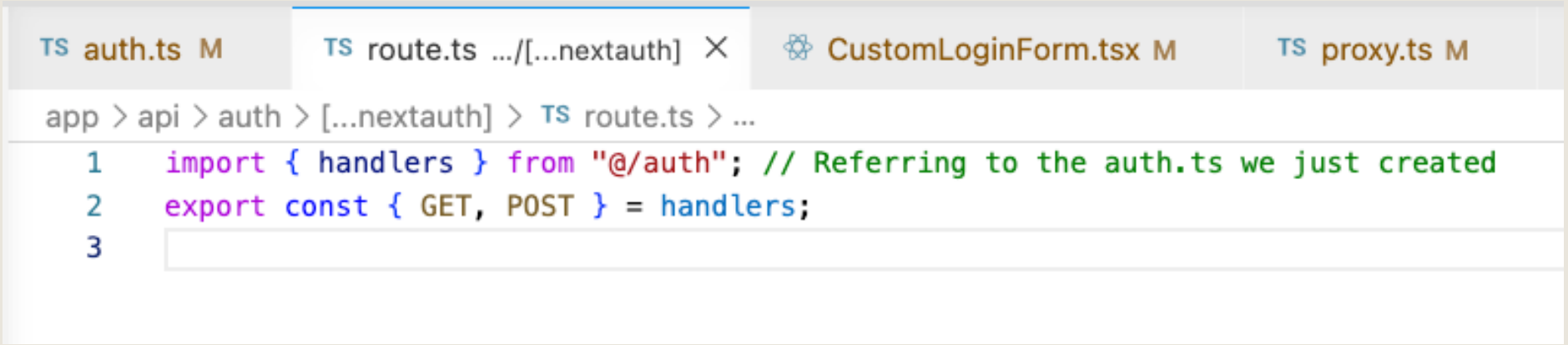
Auth.js Custom Credentials

Auth.js automatically takes care of JWT and Session!

```
43    callbacks: {
44      // Inject custom payload variables into the session token automatically
45      async jwt({ token, user }) {
46        if (user) token.id = user.id;
47        return token;
48      },
49      async session({ session, token }) {
50        if (session.user) session.user.id = token.id as string;
51        return session;
52      },
53      authorized: async ({ auth }) => {
54        // Logged in users are authenticated, otherwise redirect to login page
55        return !!auth;
56      }
57    }
58  });
```

Check API Catch-All Route

- We configured this last week for Google Provider



Modify the Client Provider

/components/Provider.tsx

Add the SessionProvider –

Can be used in NavBar component

app > components > Providers.tsx > ...

```tsx
"use client";

import { AuthProvider } from "../hooks/useAuth";
import { SessionProvider } from "next-auth/react";

export function Providers({ children }: { children: React.ReactNode }) {
  return (
    <SessionProvider>
      <AuthProvider>{children}</AuthProvider>
    </SessionProvider>
  );
}
```

Trigger Login via signIn() on signin form

Custom sign in form

Import SignIn

app > (auth) > login > CustomLoginForm.tsx > CustomLoginForm > handleAuthCredential

```
1  "use client";
2  import Link from "next/link";
3  import { useState } from "react";
4  import { useAuth } from "../../hooks/useAuth";
5  import { useRouter } from "next/navigation";
6  import { signIn } from "next-auth/react"; // Auth.js CredentialsProvider
7
```

Import checkSession from useAuth()

Code the form submit handler

Use SignIn provided by Auth.js

Call checkSession() so that it updates the state values

```tsx
const { setCustomAuthUser, checkSession } = useAuth(); // Must expose a state updater in your AuthProvider

// submit handler for Auth.js CredentialsProvider
const handleAuthCredential = async (
  e: React.SubmitEvent<HTMLFormElement>
) => {
  e.preventDefault();
  setError("");
  setIsSubmitting(true);
  try {
    // Calls Auth.js Credentials Provider
    const result = await signIn("credentials", {
      email,
      password,
      redirect: false // Prevents full page reload so we can handle errors
    });

    if (result?.error) {
      setError("Invalid credentials");
      return;
    }
    // Force your custom AuthProvider to fetch /api/auth/me and update React state & localStorage!
    await checkSession();
    // Refresh cache & redirect
    router.refresh();
    router.push("/");
  } catch (err) {
    setError("Something went wrong");
  } finally {
    setIsSubmitting(false);
  }
};
```

Trigger Login via signIn() on signin form

Update the form submit handler

Note: I use onSubmit!

```
106        )}
107        {/* <form onSubmit={handleCustomSubmit} className="mt-4"> */}
108        <form onSubmit={handleAuthCredential} className="mt-4">
109          {" "}
```

Modify our Session Handling

- /api/auth/me/route.js

app > api > auth > me > TS route.ts > GET > user

```
1  import { NextResponse } from "next/server";
2  import { cookies } from "next/headers";
3  import jwt from "jsonwebtoken";
4  import { auth } from "@/auth"; // Auth.js helper for Auth.js CredentialsProvider
```

Modify our Session Handling

- /api/auth/me/route.js

Note: make sure the JWT payload is the same as what has been set in /api/auth/login/route.ts

```
// ------------------------------------------------------------
// Check 1: Approach 1 (Custom Credentials Cookie)
// ------------------------------------------------------------
const customToken = cookieStore.get("custom_session_token")?.value;

if (customToken) {
  try {
    // Verify the JWT signature & expiration
    const secret = process.env.JWT_SECRET;
    if (!secret) {
      throw new Error("JWT_SECRET is missing");
    }

    const decoded = jwt.verify(customToken, secret) as {
      userId: string;
      email: string;
      name?: string;
    };

    return NextResponse.json({
      success: true,
      user: {
        userId: decoded.userId,
        email: decoded.email,
        name: decoded.name || decoded.email,
        provider: "conventional credentials"
      }
    });
  } catch {
    // Invalid custom token, fall through to check Auth.js
    // JWT signature invalid or expired
    return NextResponse.json({ error: "Invalid token" }, { status: 401 });
  }
}
} catch (error) {
  // JWT signature invalid or expired
  return NextResponse.json({ error: "Invalid token" }, { status: 401 });
}
```

Modify our Session Handling

- /api/auth/me/route.js

```
49  // ------------------------------------------------------------
50  // Check 2: Approaches 2 & 3 (Auth.js Credentials & Google)
51  // ------------------------------------------------------------
52
53  const session = await auth();
54
55  if (session?.user) {
56    return NextResponse.json({
57      authenticated: true,
58      user: {
59        id: session.user.id || session.user.email,
60        email: session.user.email,
61        name: session.user.name || session.user.email,
62        // Auth.js handles both credentials and google under its session
63        provider: "authjs"
64      }
65    });
66  }
```

Modify our Session Handling

- /api/auth/me/route.js

```
67
68    // ------------------------------------------------------------
69    // Unauthenticated Fallback
70    // ------------------------------------------------------------
71    return NextResponse.json(
72      { authenticated: false, user: null },
73      { status: 401 }
74    );
```

Don’t forget the NavBar. Modify useAuth()

- Final version!

app > hooks > useAuth.tsx > AuthContextType > checkGoogle

```tsx
"use client";

import {
  createContext,
  useContext,
  useState,
  useEffect,
  useCallback,
  ReactNode
} from "react";

import {
  checkGoogleSession,
  handleGoogleSignout
} from "../actions/GoogleAction";

import { signOut as authJsSignOut } from "next-auth/react"; // for CredentialsProvider
```

Don’t forget the NavBar. Modify useAuth()

- Final version!

```
19  interface User {
20    id: string | null | undefined;
21    email: string | null | undefined;
22    name: string | null | undefined;
23    provider?: "conventional credentials" | "authjs" | "google" | string; // track login provider
24  }
25
```

```
47  interface AuthContextType extends AuthState {
48    checkGoogle: () => void;
49    register: (credentials: RegisterCredentials) => Promise<AuthResponse>;
50    setCustomAuthUser: (user: User) => void;
51    logout: () => Promise<void>;
52    checkSession: () => Promise<void>;
53  }
```

Don’t forget the NavBar. Modify useAuth()

- Final version!

```
54  const checkGoogle = async () => {
55    const session = await checkGoogleSession();
56    if (session) {
57      const currentUser: User = {
58        id: session.user?.id,
59        name: session.user?.name,
60        email: session.user?.email,
61        provider: "google"
62      };
63      localStorage.setItem("custom_user", JSON.stringify(currentUser));
64      setAuthState({
65        user: currentUser,
66        isAuthenticated: true,
67        isLoading: false
68      });
69    }
70  };
```

- Final version!

  Check all sessions and update the state values

```
const checkSession = useCallback(async () => {
  try {
    // 1. Verify Custom JWT Cookie against the server endpoint
    const res = await fetch("/api/auth/me");
    if (res.ok) {
      const data = await res.json();
      if (data.user) {
        const userData: User = {
          id: data.user.id,
          email: data.user.email,
          name: data.user.name || data.user.email,
          provider: data.user.provider || "conventional credentials"
        };
        //Save to localStorage regardless of login provider
        localStorage.setItem("custom_user", JSON.stringify(userData));
        setAuthState({
          user: userData,
          isAuthenticated: true,
          isLoading: false
        });
        return; // Stop here: Custom credentials session verified!
      }
    }

    // 2. Fallback: Check Google Session
    const googleSession = await checkGoogleSession();

    if (googleSession?.user) {
      const userData: User = {
        id: googleSession.user.id,
        name: googleSession.user.name,
        email: googleSession.user.email,
        provider: "google"
      };
      // Save to localStorage
      localStorage.setItem("custom_user", JSON.stringify(userData));
      setAuthState({
        user: userData,
        isAuthenticated: true,
        isLoading: false
      });
      return;
    }

    // No active session -> Clear state & local storage
    localStorage.removeItem("custom_user");
    setAuthState({
      user: emptyUser,
      isAuthenticated: false,
      isLoading: false
    });
  } catch (error) {
    localStorage.removeItem("custom_user");
    setAuthState({
      user: emptyUser,
      isAuthenticated: false,
      isLoading: false
    });
  }
}, []);
```

Don’t forget the NavBar. Modify useAuth()

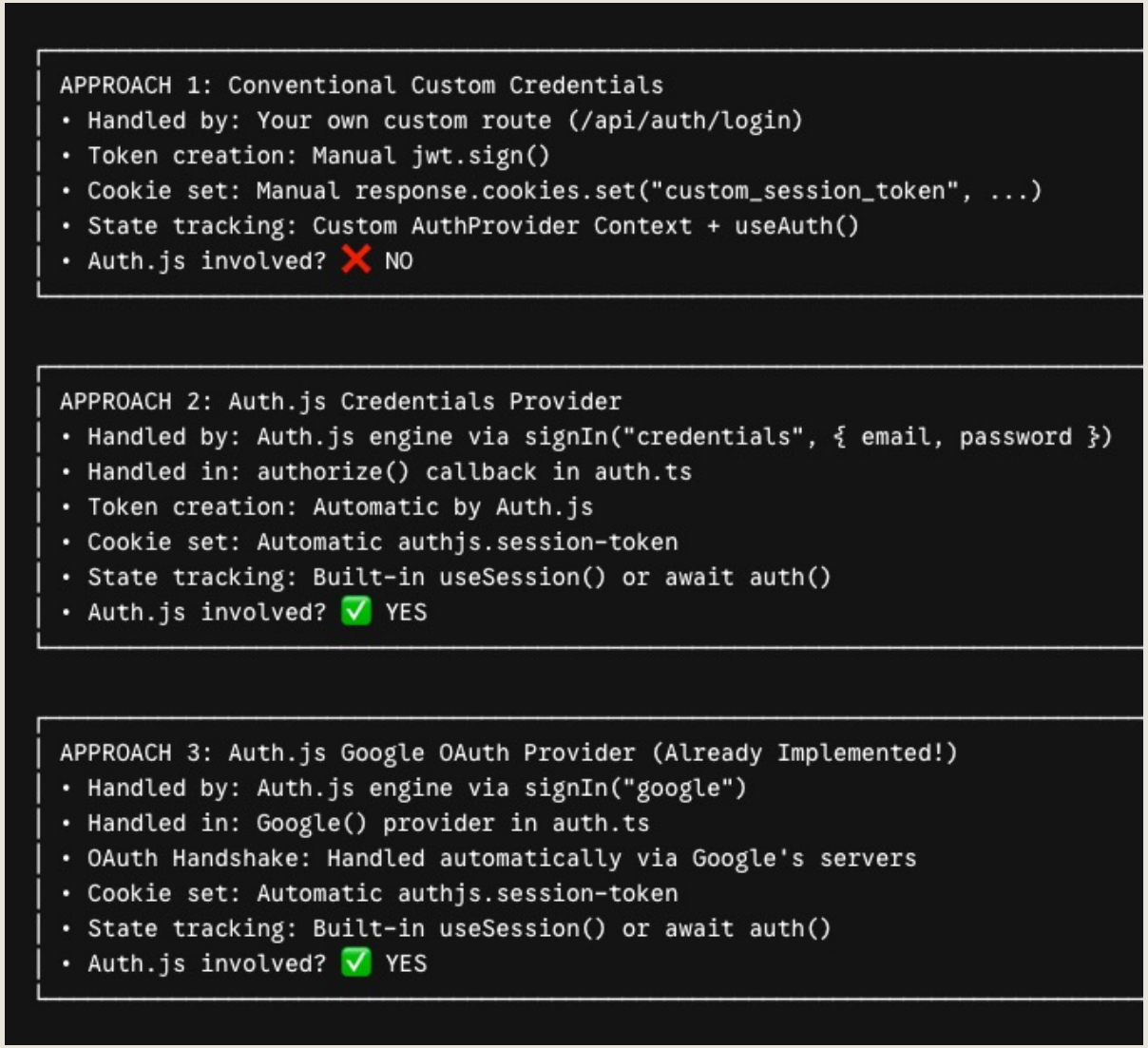
- Final version!

Logout

```
148
149   const logout = useCallback(async () => {
150     try {
151       // 1. Clear Custom Credentials JWT Cookie on the server
152       await fetch("/api/auth/logout", { method: "POST" });
153
154       // 2. Clear Google Session (if Google signout action exists)
155       await handleGoogleSignout();
156       await authJsSignOut({ redirect: false }); // for CredentialsProvider
157     } catch (error) {
158       console.error("Logout error:", error);
159     } finally {
160       // 3. CLEAR LOCAL STORAGE
161       // Option A: Remove specific auth keys (Recommended)
162       localStorage.removeItem("custom_user");
163
164       // 4. Reset React State
165       setAuthState({
166         user: emptyUser,
167         isAuthenticated: false,
168         isLoading: false
169       });
170     }
171   }, []);
```

Test it!

- Conventional Custom Credentials
- Auth.js Credential Provider
- Google Provider



Custom vs Auth.js

Lines of Code & Infrastructure Handshake:

- **Custom:** We own token creation, cookie flag declarations (HttpOnly, SameSite, Path), error payloads, and API routing.
- **Auth.js:** We delegate HTTP cookie headers, CSRF tokens, session updates, and client hooks (useSession()) directly to the framework.

Custom vs Auth.js

Error Safety:

- **Custom:** Omitting httpOnly: true or sameSite: "lax" exposes users to XSS token theft or CSRF attacks.
- **Auth.js:** Sets hardened security headers and encrypted cookie rules out of the box.

Custom vs Auth.js

Session Strategy Locking:

- **Custom:** Can switch between stateful database sessions (e.g. SQL session table) or stateless JWTs easily.
- **Auth.js:** `CredentialsProvider` uses **JWT session strategies** to minimise database load and security anti-patterns.

Auth.js Credentials Provider

To discourage us from building raw password systems when safer options exist, Auth.js imposes friction on the `CredentialsProvider`:

- It does **not** manage password hashing or user creation.
- It does **not** auto-sync credentials users into database tables.
- It forces us to explicitly own the authorize() logic and password storage!

When we attach a **Database Adapter** to Auth.js (like Prisma or MongoDB adapters), Auth.js assumes a **stateful database session model**

Securing the App

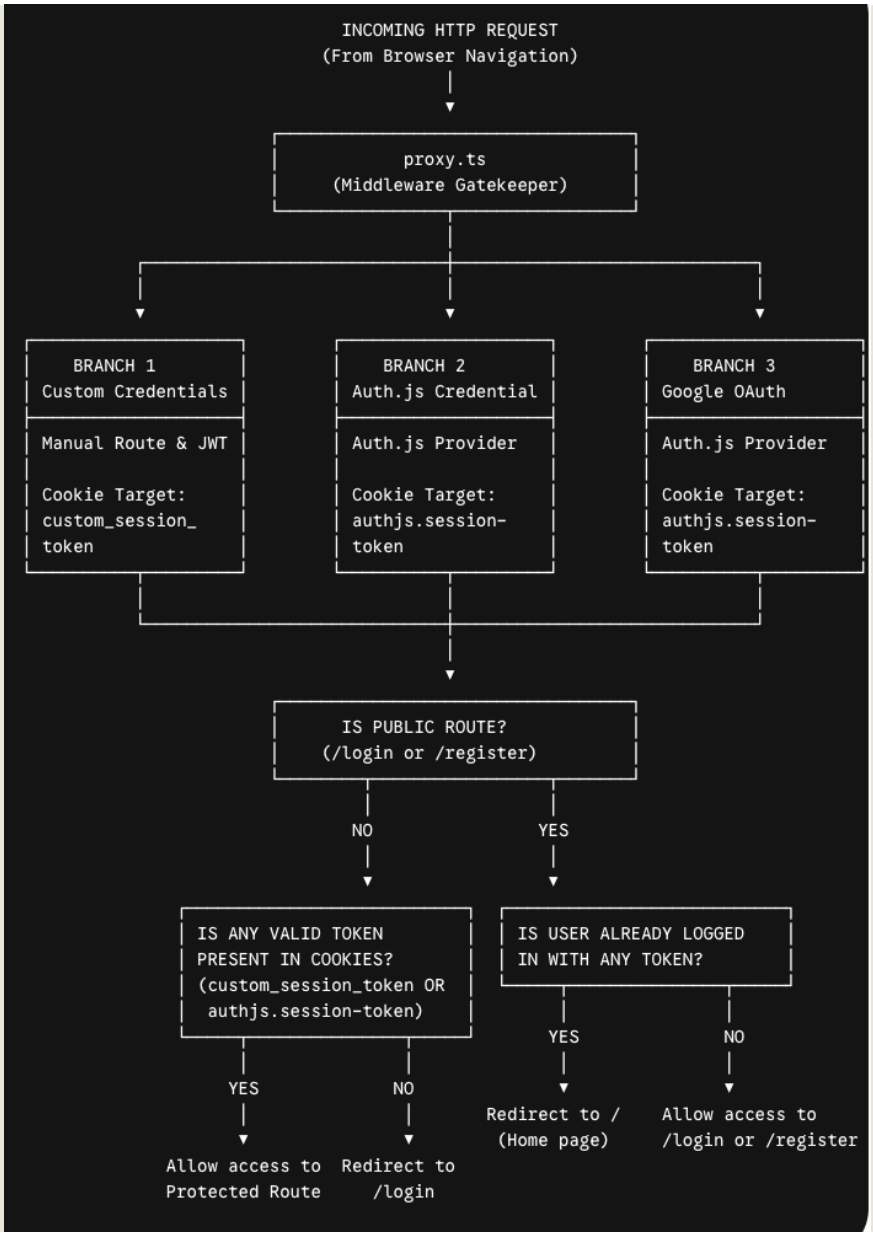
Securing API Routes

Robust Architecture

Unified Gatekeeper: `proxy.ts` acts as the single security check for all incoming requests before any page renders.

Provider Agnostic: It doesn't care whether the user logged in via manual JWT, Auth.js Credentials, or Google—it simply asks: *"Is there a valid cookie present?"*

Smooth UX: Guests can freely access /login and /register, while logged-in users are automatically kept away from auth forms and sent straight to the main app.



Securing API Routes with Robust Architecture

Use middleware (proxy.ts) – final version!

```ts
export function proxy(req: NextRequest) {
  const { pathname } = req.nextUrl;

  // 1. Extract the Approach 1 custom cookie
  const customToken = req.cookies.get("custom_session_token")?.value;
  const hasCustomAuth =
    typeof customToken === "string" && customToken.trim().length > 0;

  // 2. Check for Approach 2 & 3: Auth.js / NextAuth Session Cookies (Google & Auth.js Credentials)
  const authJsToken =
    req.cookies.get("authjs.session-token")?.value ||
    req.cookies.get("__Secure-authjs.session-token")?.value ||
    req.cookies.get("next-auth.session-token")?.value ||
    req.cookies.get("__Secure-next-auth.session-token")?.value;

  const hasAuthJsAuth =
    typeof authJsToken === "string" && authJsToken.trim().length > 0;

  // Ensure token exists AND is a non-empty string
  // const isAuthenticated =
  //   typeof customToken === "string" && customToken.trim().length > 0;

  // Combined Auth Check: User is logged in if ANY valid token exists
  const isAuthenticated = hasCustomAuth || hasAuthJsAuth;

  // 2. Define public routes that anyone can visit (Login and Register)
  const isPublicRoute =
    pathname.startsWith("/login") || pathname.startsWith("/register");

  // console.log(isAuthenticated);
  // console.log(isPublicRoute);

  // 3. UNAUTHENTICATED USER trying to access protected page -> Bounce to /login
  if (!isAuthenticated && !isPublicRoute) {
    return NextResponse.redirect(new URL("/login", req.nextUrl.origin));
  }

  // 4. AUTHENTICATED USER trying to visit /login or /register -> Bounce to home /
  if (isAuthenticated && isPublicRoute) {
    return NextResponse.redirect(new URL("/", req.nextUrl.origin));
  }

  return NextResponse.next();
}
```

Timing Attack Mitigation

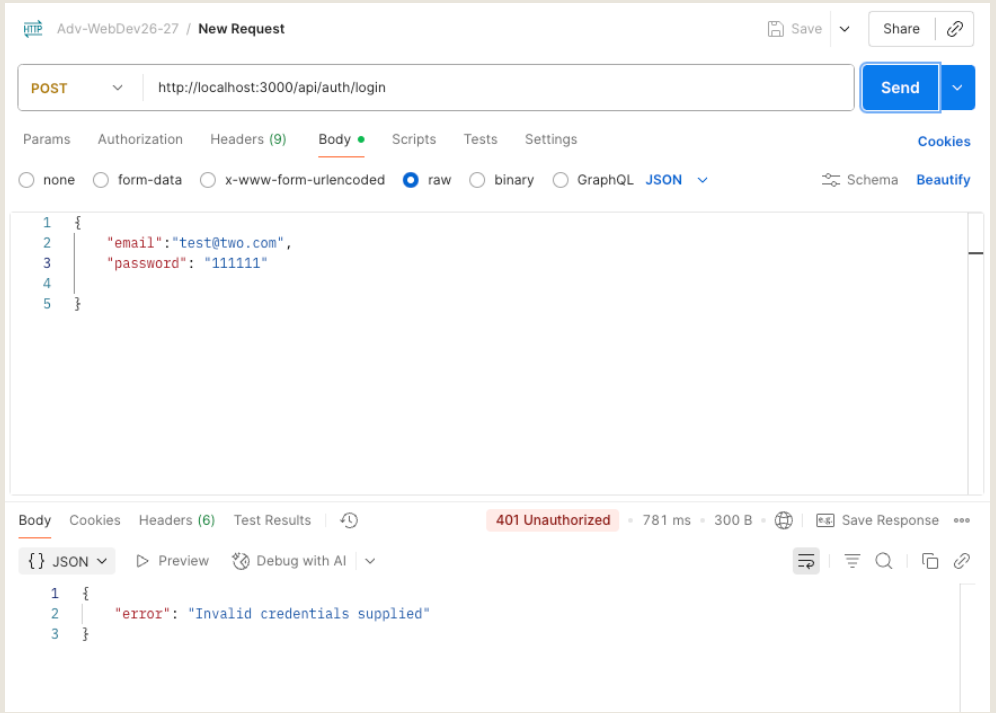
- You can add this immediately without setting up any third-party services.
- Update your database check to run a "dummy" hash comparison if the user isn't found.
- This ensures your API always takes the exact same amount of time to respond, preventing attackers from guessing which emails exist in your system.

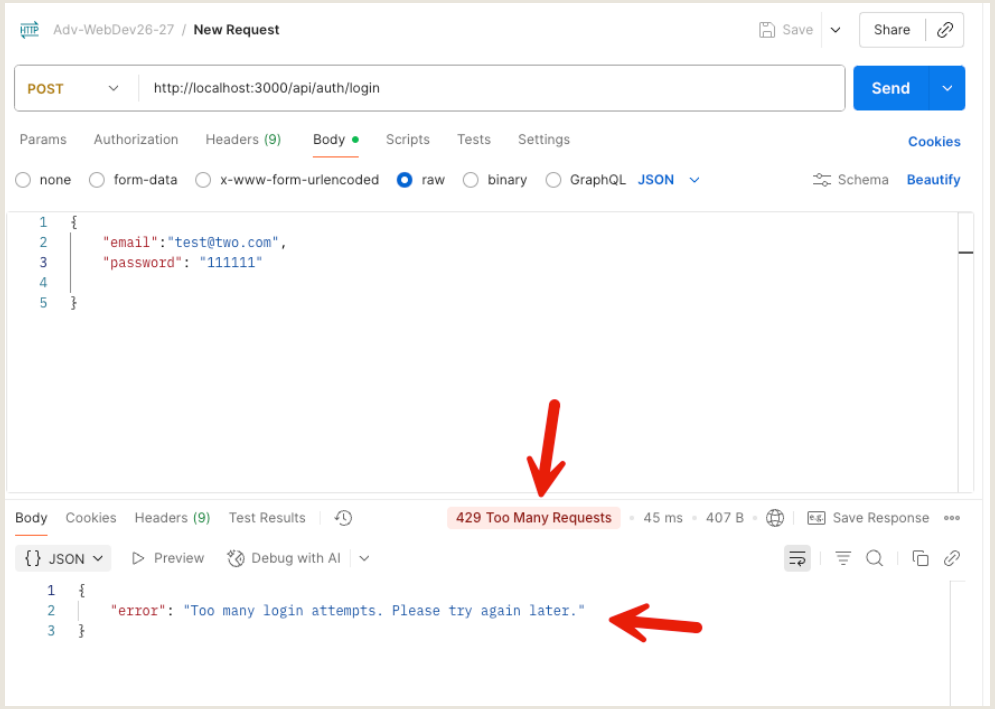
api/auth/login/route.ts



```
48
49      // 4. Fetch user from database
50      const user = await prisma.user.findUnique({ where: { email } });
51      if (!user) {
52        // Run a dummy comparison against a random bcrypt hash so the response time is identical
53        await bcrypt.compare(password, "$2b$10$abcdefghijklmnopqrstuv");
54        return NextResponse.json(
55          { error: "Invalid credentials supplied" },
56          { status: 401 },
57        );
58      }
59
```

What about traffics bypass the frontend?





Redis Rate Limiting

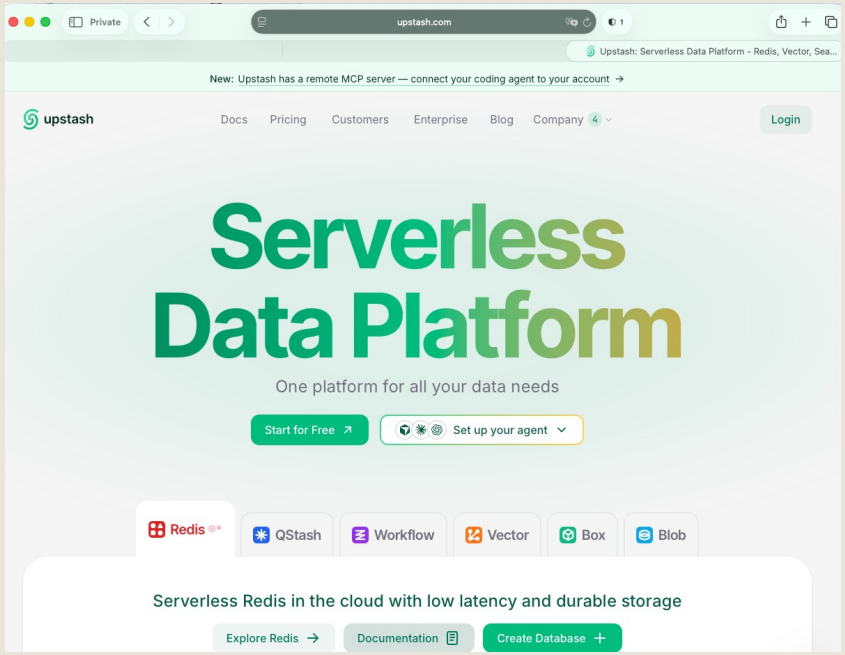
- permanently stop brute-force bots and script kiddies from spamming your database.
- The easiest and most serverless-friendly way to do this in Next.js is using `Upstash Redis`, as it works perfectly over HTTP and does not require persistent connections.

Upstash Redis

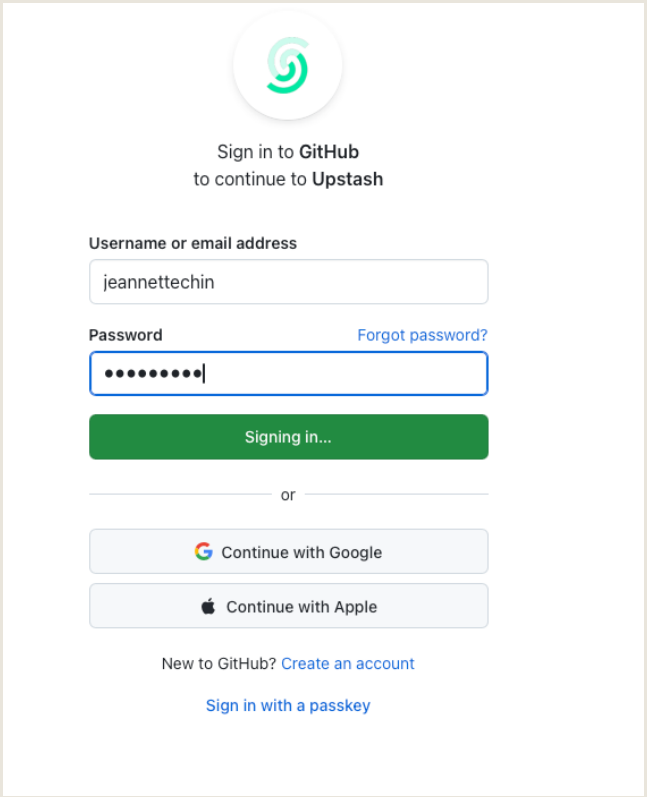
- Install

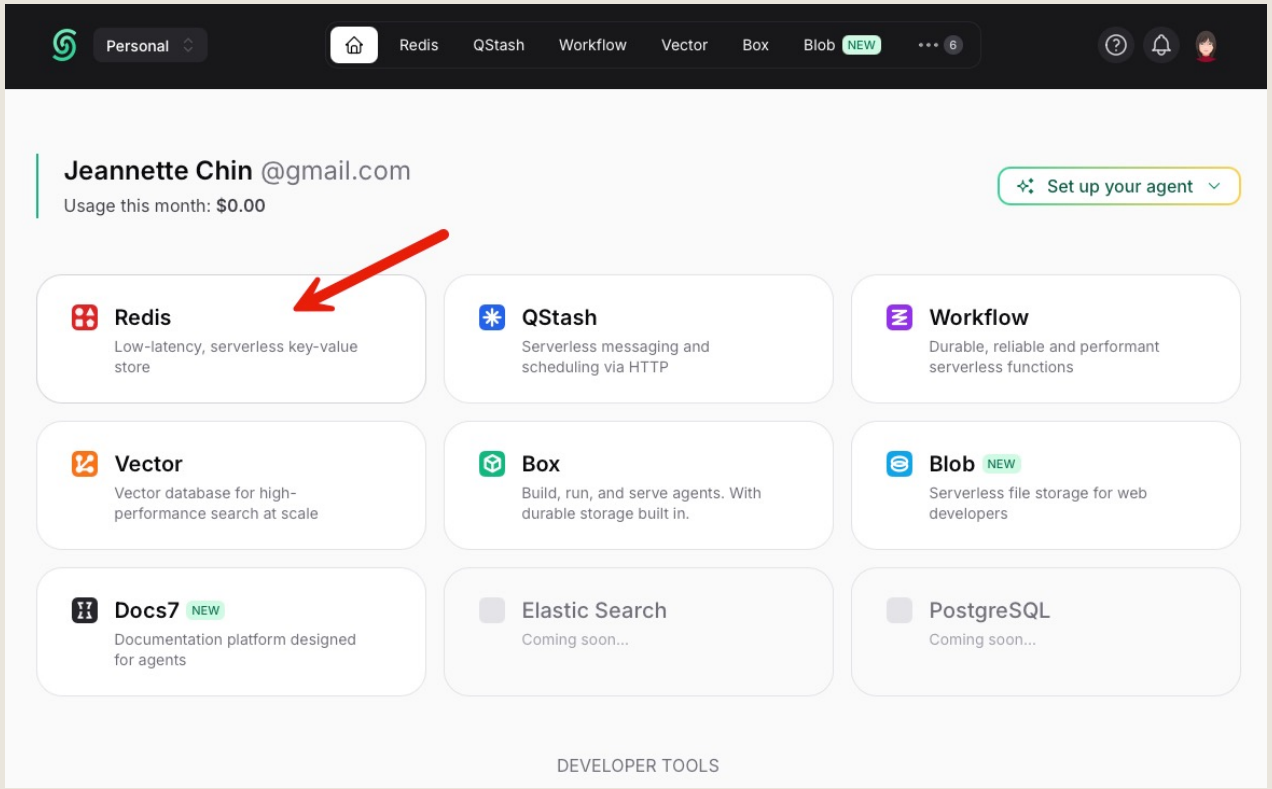
npm install @upstash/redis @upstash/ratelimit

Create a free Redis database at Upstash

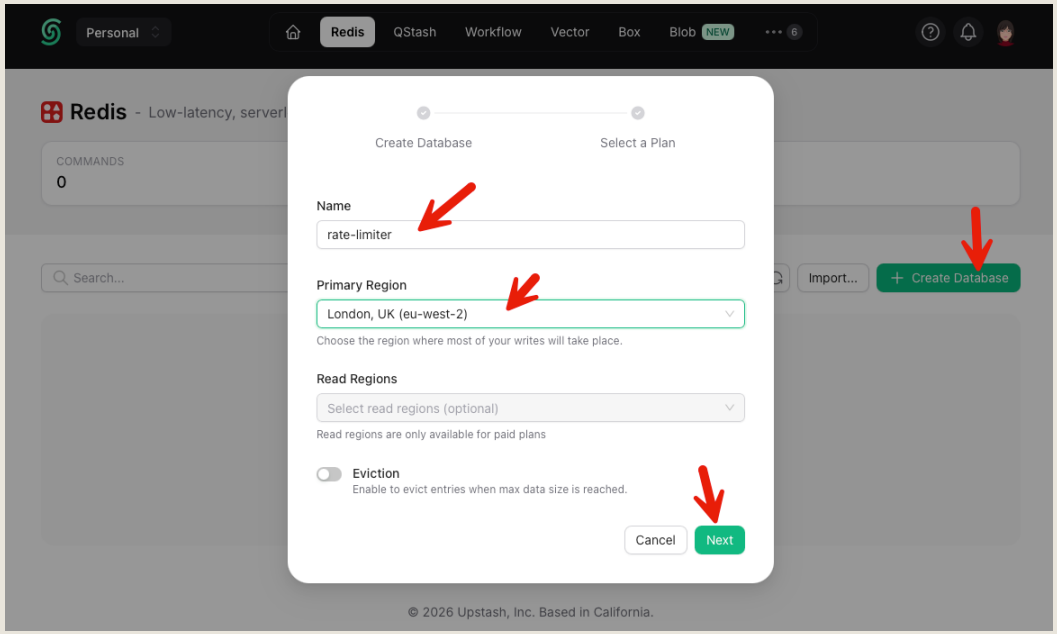


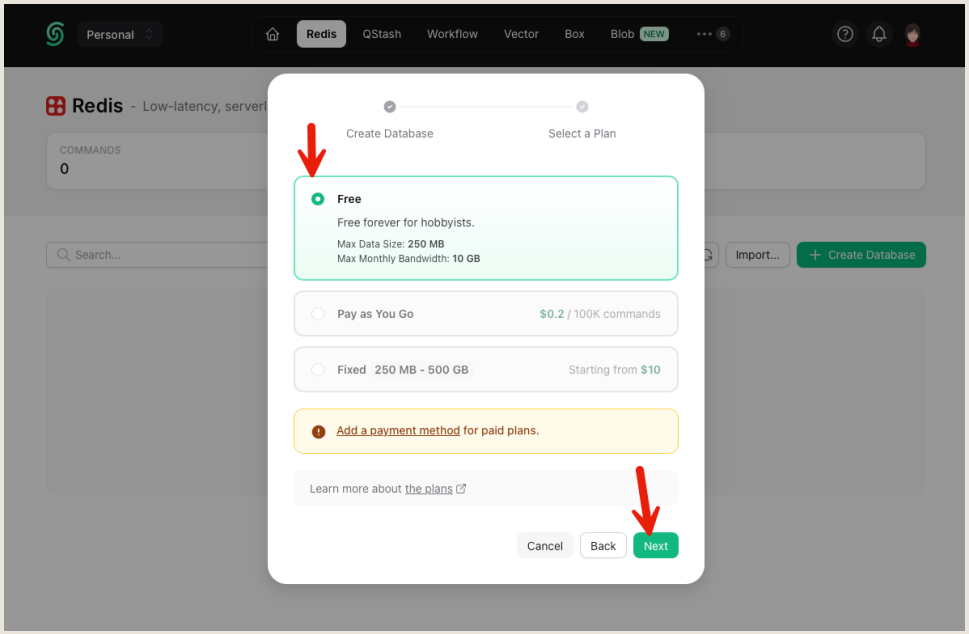
Upstash Redis



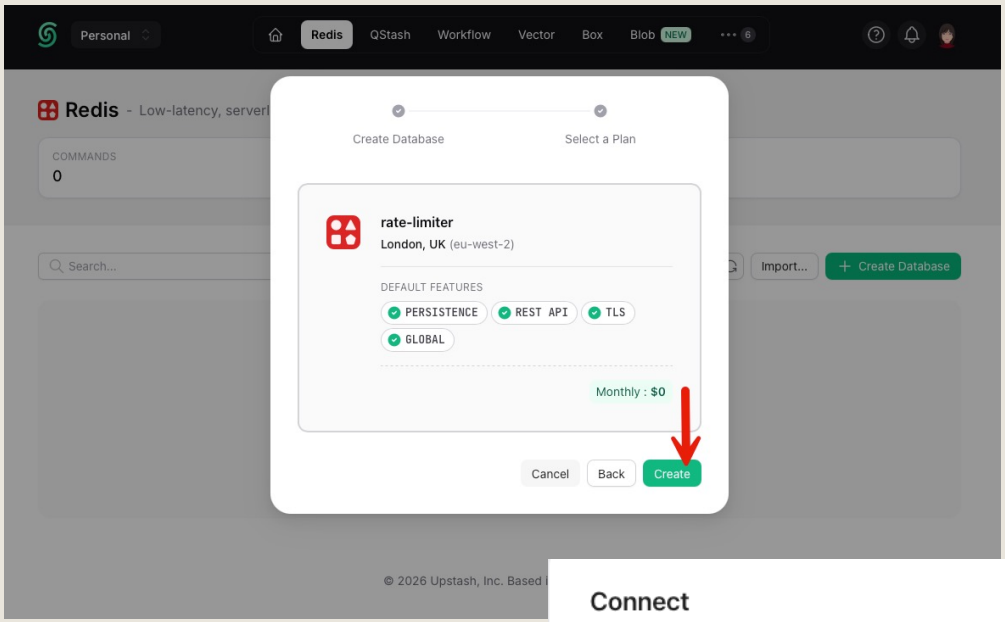


Upstash Redis





Upstash Redis





Upstash Redis

- Paste the credentials to your .env file



Upstash Redis

- Api/auth/login/route.ts

app > api > auth > login > TS route.ts > POST

```ts
import { NextResponse } from "next/server";
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
import { prisma } from "../../../lib/prisma";
import { Redis } from "@upstash/redis";
import { Ratelimit } from "@upstash/ratelimit";

// Initialise Redis and the Rate Limiter (e.g., 5 requests per 15 minutes)
const redis = Redis.fromEnv();
const ratelimit = new Ratelimit({
  redis: redis,
  limiter: Ratelimit.slidingWindow(5, "15 m"),
  analytics: true,
});
```

- Limit 5 attempts
- Wait for 15 minutes

Upstash Redis

- Api/auth/login/route.ts

```
16  export async function POST(req: Request) {
17    try {
18      // 1. Extract the user's IP address
19      const ip = req.headers.get("x-forwarded-for") ?? "127.0.0.1";
20
21      // 2. Check the rate limit
22      const { success, pending, limit, reset, remaining } =
23        await ratelimit.limit(ip);
24
25      if (!success) {
26        return NextResponse.json(
27          { error: "Too many login attempts. Please try again later." },
28          {
29            status: 429,
30            headers: {
31              "X-RateLimit-Limit": limit.toString(),
32              "X-RateLimit-Remaining": remaining.toString(),
33              "X-RateLimit-Reset": reset.toString(),
34            },
35          },
36        );
37      }
38
```

Practical Exercise and homework

- Lab 5 – Implementing Custom Credentials Authentication Systems and with Auth.js

Next Week – Independent Study Week

- Part 1 of the module is now completed
- No lecture next week!
- Lab sessions run as usual (drop in, no new learning materials)

001 Assignment due on Friday Week 7 -> 6 November 2026

In person ViVa is scheduled in Week 8 (book your slot)

Very Best of Luck!

Feedback



QR